

Nível de Aplicação

**Licenciatura em Engenharia
de Telecomunicações e Informática**

3º ano - 2º Semestre

2024/2025

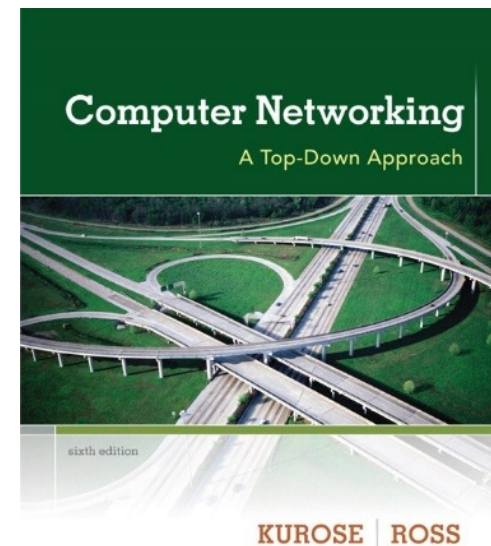


Sumário



- Conceitos gerais
- Paradigmas do nível de aplicação:
 - Paradigma cliente-servidor
 - Paradigma peer-to-peer
- Alguns protocolos do nível de aplicação
 - HTTP, FTP, SMTP/POP3/IMAP, DNS

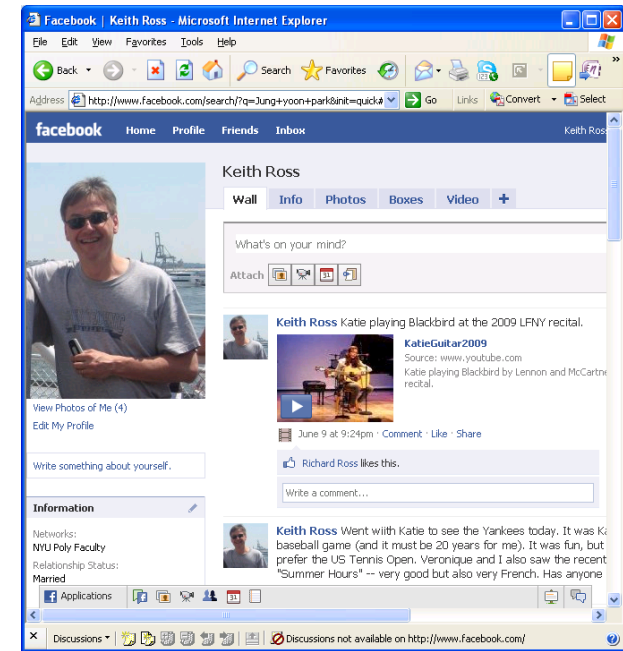
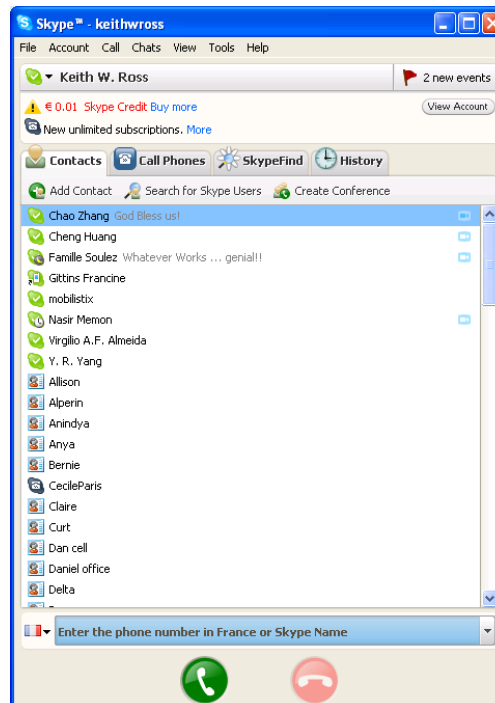
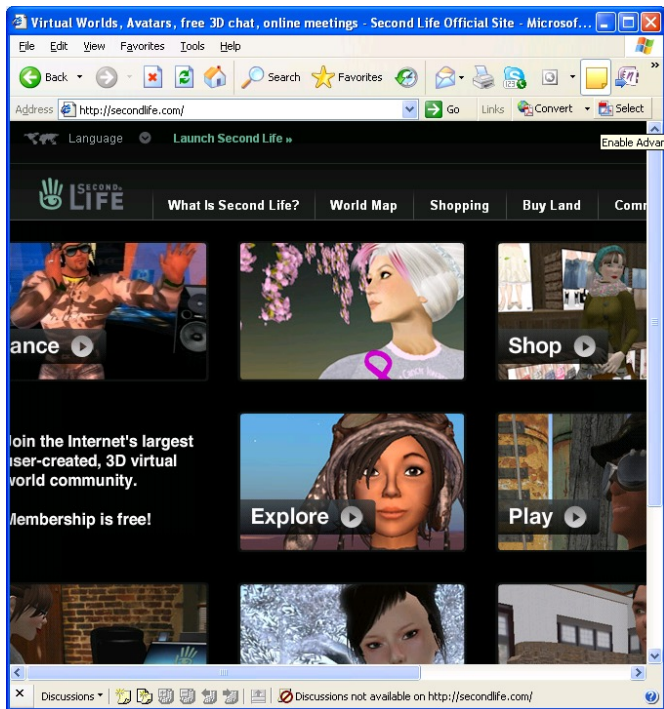
Estes slides são maioritariamente baseados no livro: *Computer Networking: A Top-Down Approach Featuring the Internet, Jim Kurose and Keith Ross, Addison-Wesley*



Algumas aplicações de rede



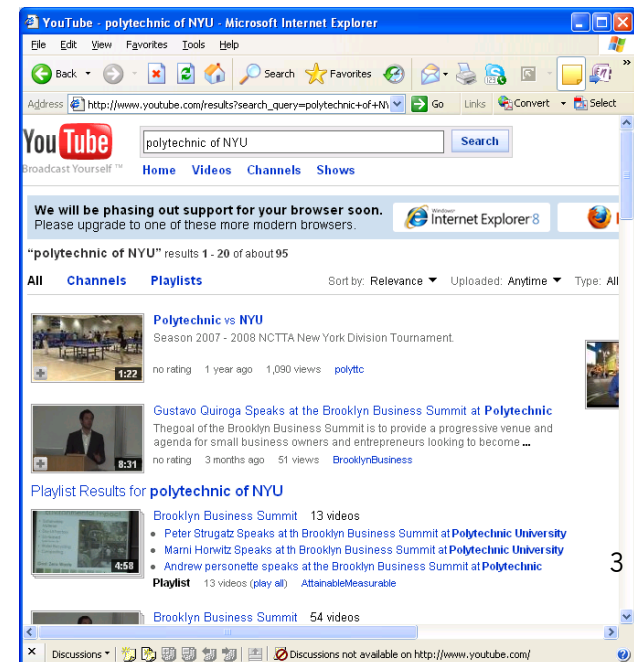
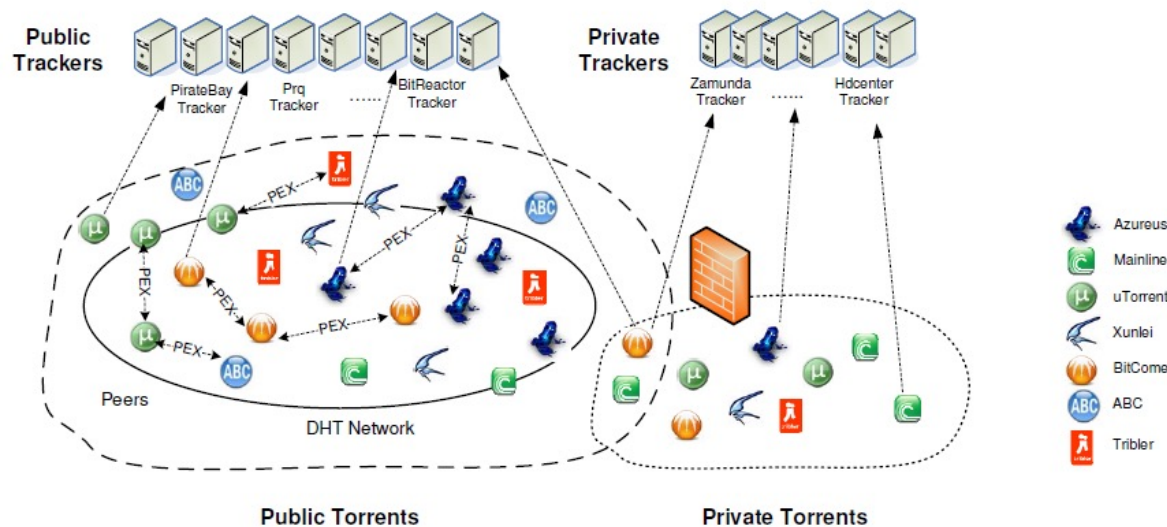
- **e-mail**
- **web**
- **instant messaging**
- **acesso remoto**
- **P2P file sharing**
- **jogos em rede**
- **clips de video**
- **redes sociais**
- **voice over IP**
- **vídeo conferencia em tempo-real**
- **grid computing**



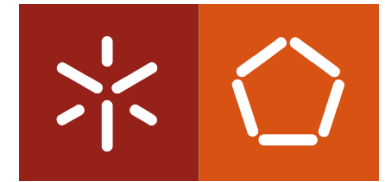
Public
Discovery
Sites



Private
Discovery
Sites



Criar uma aplicação em rede

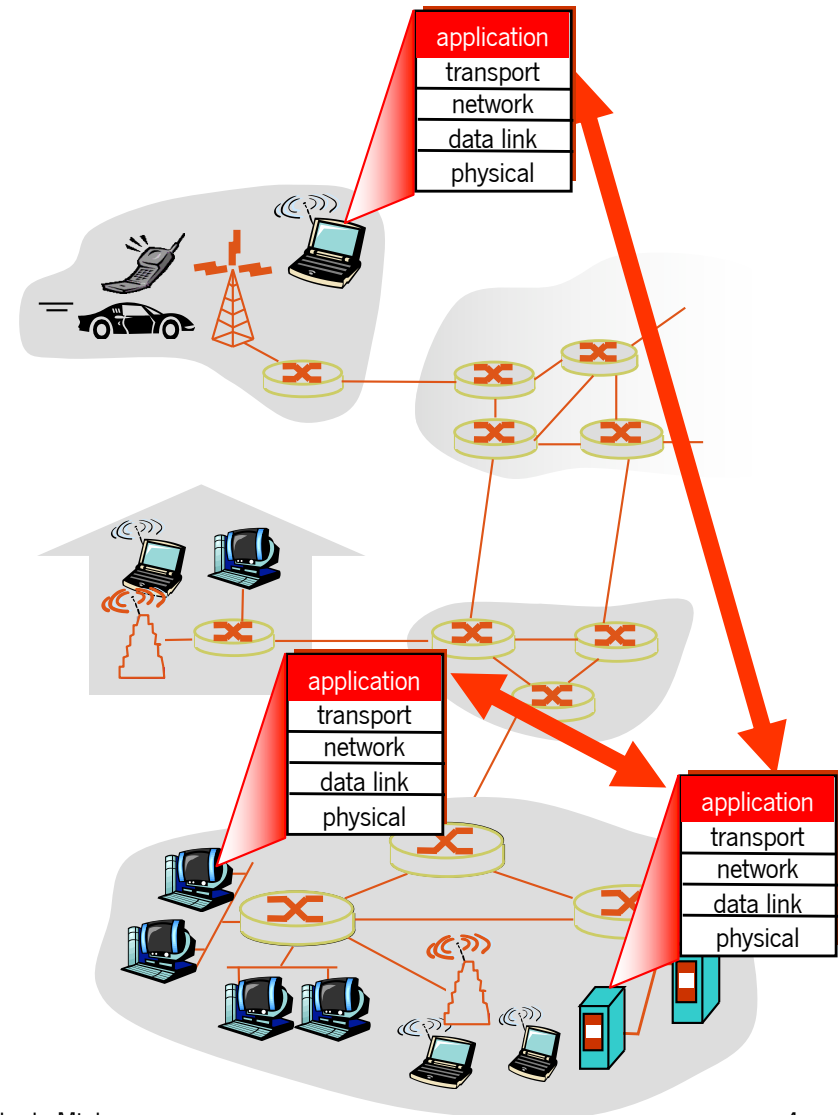


Desenvolver um programa que:

- corra em diferentes sistemas terminais
- comunique através da rede
- ex., o software do servidor web comunica com o software do browser

Não é preciso desenvolver software para os dispositivos do núcleo da rede:

- Os dispositivos do núcleo da rede não correm aplicações para o utilizador
- O facto das aplicações estarem hospedadas nos sistemas terminais permite um rápido desenvolvimento e propagação

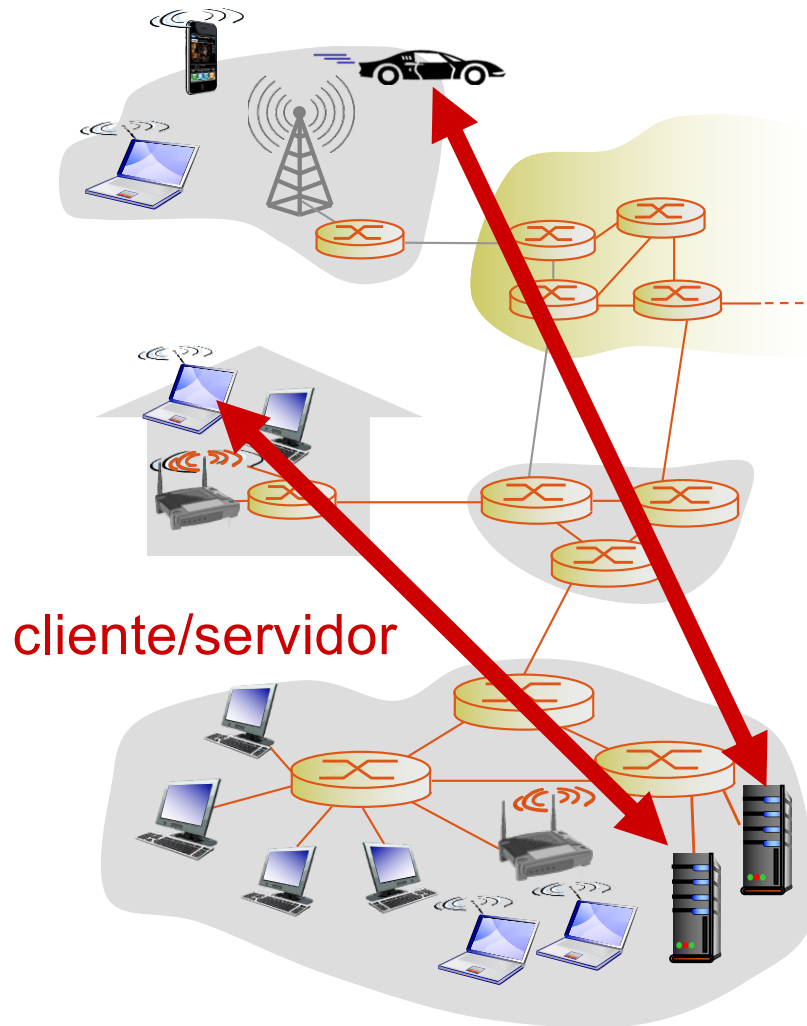
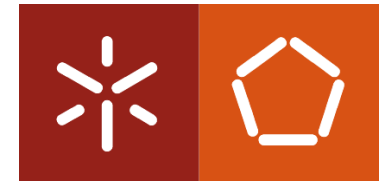


Arquitecturas do nível de aplicação



- **Cliente-servidor**
- **Peer-to-peer (P2P)**
- **Híbridas: cliente-servidor e peer-to-peer**

Arquitetura Cliente-Servidor



Servidor:

- Sempre ligado
- Com um endereço IP permanente
- Data centers para “escalar”

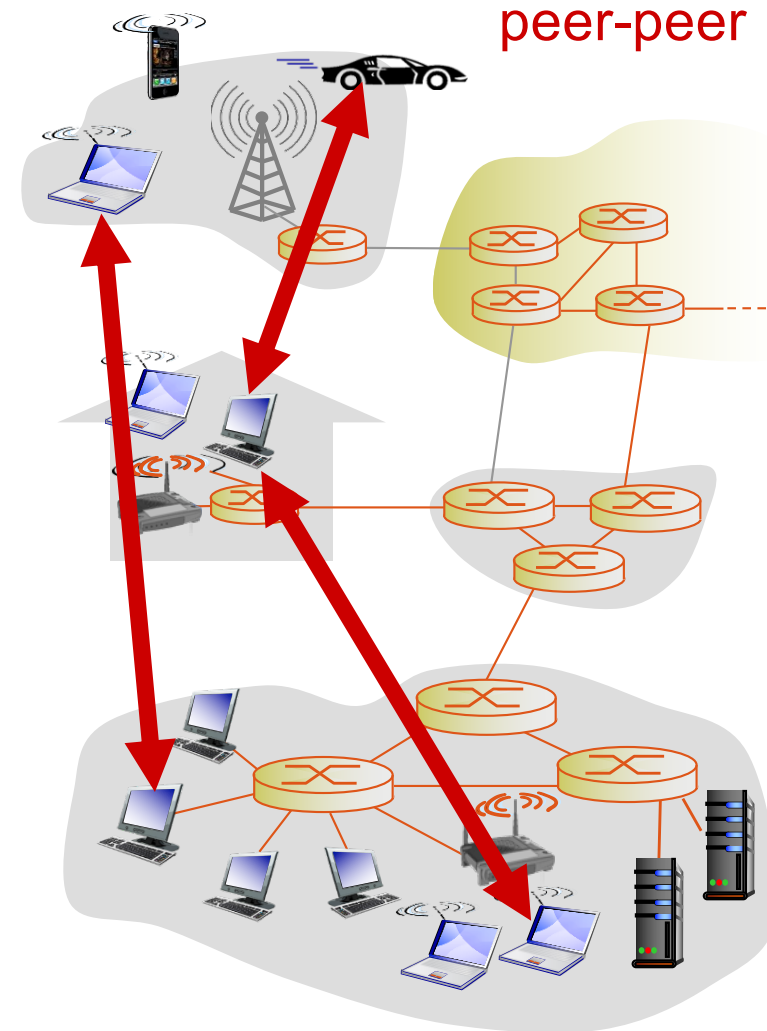
Clientes:

- Comunicam com o servidor
- Podem estar ligados de forma intermitente
- Podem possuir um endereço IP dinâmico
- Não comunicam diretamente uns com os outros

Arquitetura P2P



- Não há servidores permanentemente ligados
- Os peers comunicam diretamente uns com os outros.
- Requerem serviços de peers e em troca disponibilizam serviços a outros peers.
 - *escalável – novos peers trazem uma maior capacidades ao serviço, assim como novas necessidades.*
- Os peers estão ligados de forma intermitente e mudam frequentemente de endereço IP
- Gestão complexa



Híbridos: cliente-servidor e P2P



- **Skype**
 - Aplicação voice-over-IP P2P
 - Servidor centralizado: procura o endereço da parte remota
 - Conexão cliente-cliente: directa (não através do servidor)
- **Instant messaging**
 - A conversa entre dois utilizadores é P2P
 - Serviço centralizado: a presença e a localização do cliente é detectada
 - O utilizador regista o seu IP num servidor centralizado quando está online
 - O utilizador contacta o servidor central para encontrar o endereço IP da pessoa com quem quer manter uma conversa

Requisitos do serviço de transporte para as aplicações mais comuns



Aplicação	Perda de dados	Taxa de transmissão	Sensibilidade ao atraso
Trans. de ficheiros	não	elástica	não
e-mail	não	elástica	não
Páginas Web	não	elástica	não
áudio/vídeo em tempo-real	tolerante	áudio: 5kbps-1Mbps vídeo: 10kbps-5Mbps	sim, 100's mseg
<i>streaming</i> áudio/vídeo	tolerante	idem	sim, poucos seg
Jogos interactivos	tolerante	poucos kbps	sim, 100 mseg
Mens. instantâneas	não	elástica	sim e não

Aplicações de Internet: protocolos de aplicação e de transporte



Aplicação	Protocolo do nível de aplicação	Protocolo de transporte de suporte
e-mail	SMTP [RFC 2821]	TCP
Acesso remoto	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
Transferência de fich. streaming multimedia	FTP [RFC 959]	TCP
	HTTP (ex Youtube), RTP [RFC 1889]	TCP or UDP
VoIP	SIP, RTP, proprietário (ex. Skype)	tipicamente UDP

HTTP

Hypertext Transfer Protocol



Conceitos básicos, bem conhecidos...

- Uma **página Web** consiste numa coleção de objetos
- Um objeto pode ser um ficheiro HTML, uma imagem JPEG image, um applet Java, um ficheiro audio...
- Uma página Web consiste num ficheiro de base **HTML** que inclui várias referências a outros objetos
- Cada objeto é endereçado por uma **URL (Uniform Resource Locator)**

URL exemplo:

`http://www.di.uminho.pt/cursos/miei.html`

host name

path name

HTTP

Como funciona?

HTTP: hypertext transfer protocol

- **Protocolo do nível da aplicação**
- **Modelo cliente/servidor**
 - *cliente*: browser pede, recebe e mostra objetos Web
 - *servidor*: servidor envia objetos como resposta a pedidos

HTTP 1.0: RFC 1945 (maio 1996)

HTTP 1.1: RFC 2068 (janeiro 1997)

HTTP 2.0: RFC 7540 (maio 2015)

HTTP 3.0: RFC 9114 (junho 2022)

PC a executar o
Firefox



Pedido HTTP

Resposta HTTP

Pedido HTTP

Resposta HTTP



Mac a executar o
Safari



Servidor a
executar
o servidor
WEB Apache

HTTP

Como funciona?



Utiliza o TCP:

- O cliente inicia uma conexão TCP (cria um *socket*) com um servidor HTTP (porta 80).
- O servidor TCP aceita o pedido de conexão do cliente
- São trocadas mensagens HTTP (mensagens de protocolo de nível de aplicação) entre o browser (cliente HTTP) e o servidor Web (servidor HTTP)
- A ligação TCP é terminada

O HTTP não tem estado

- O servidor não mantém estado acerca dos pedidos anteriores dos clientes

Os protocolos orientados ao estado são mais complexos!

- O passado tem que ser armazenado
- Se o servidor/cliente falham a sua visão do estado pode ficar inconsistente e terá que ser sincronizada

HTTP

Tipos de ligações



HTTP não persistente

- Só pode ser enviado no máximo um objeto Web por cada conexão estabelecida
- O HTTP/1.0 utiliza HTTP não persistente

HTTP persistente

- Podem ser enviados múltiplos objetos Web por cada ligação estabelecida entre o cliente e o servidor.
- O HTTP/1.1 usa por defeito conexões persistentes



HTTP não persistente:

- exige 2 RTTs por objecto
- O Sistema Operativo tem que reservar recursos para cada ligação TCP estabelecida
- Muitos browsers abrem ligações TCP paralelas para irem buscar os objectos referenciados

HTTP persistente:

- O servidor deixa a ligação aberta depois de enviar a mensagem de resposta
- Os pedidos HTTP posteriores são enviados através da mesma ligação

Persistente sem pipelining:

- O cliente envia um novo pedido apenas quando recebe a resposta ao anterior
- Um RTT por cada objeto referenciado

Persistente com pipelining:

- Modo por defeito no HTTP/1.1
- O cliente envia os pedido assim que os encontra no objecto referenciador
- No mínimo é consumido um RTT por todos os objetos referenciados

HTTP

Não persistente



Supondo que o utilizador introduziu a url `www.uminho.pt/DSI/index.html`

(contém texto e referência
para imagens jpeg)

1a. O cliente HTTP inicia **uma conexão TCP** com o servidor HTTP que está a ser executado no sistema `www.uminho.pt` e está à escuta na porta 80

1b. O servidor HTTP que está a ser executado no sistema `www.uminho.pt` e está à **escuta na porta 80** aceita o pedido de conexão e avisa o cliente

2. O cliente HTTP envia uma mensagem HTTP do tipo ***request message*** (contendo a URL) através de um novo socket TCP. A mensagem indica que o cliente deseja o objecto Web **`DSI/index.html`**

3. O servidor HTTP recebe a ***request message*** e **constrói uma *response message*** que contém o objeto Web requerido, enviando depois essa mensagem através do socket TCP estabelecido

tempo

HTTP

Não persistente



5. O cliente HTTP **recebe a response message** que contem o ficheiro html, mostra o ficheiro e faz o *parsing* do seu conteúdo encontrando a referência a vários objetos jpeg
 6. Repete os passos 1-5 para cada objecto referenciado
4. O servidor HTTP pede **para terminar a conexão**, mas a ligação só é terminada quando o cliente receber a response message
- 

tempo



HTTP

Modelo do Tempo de Resposta

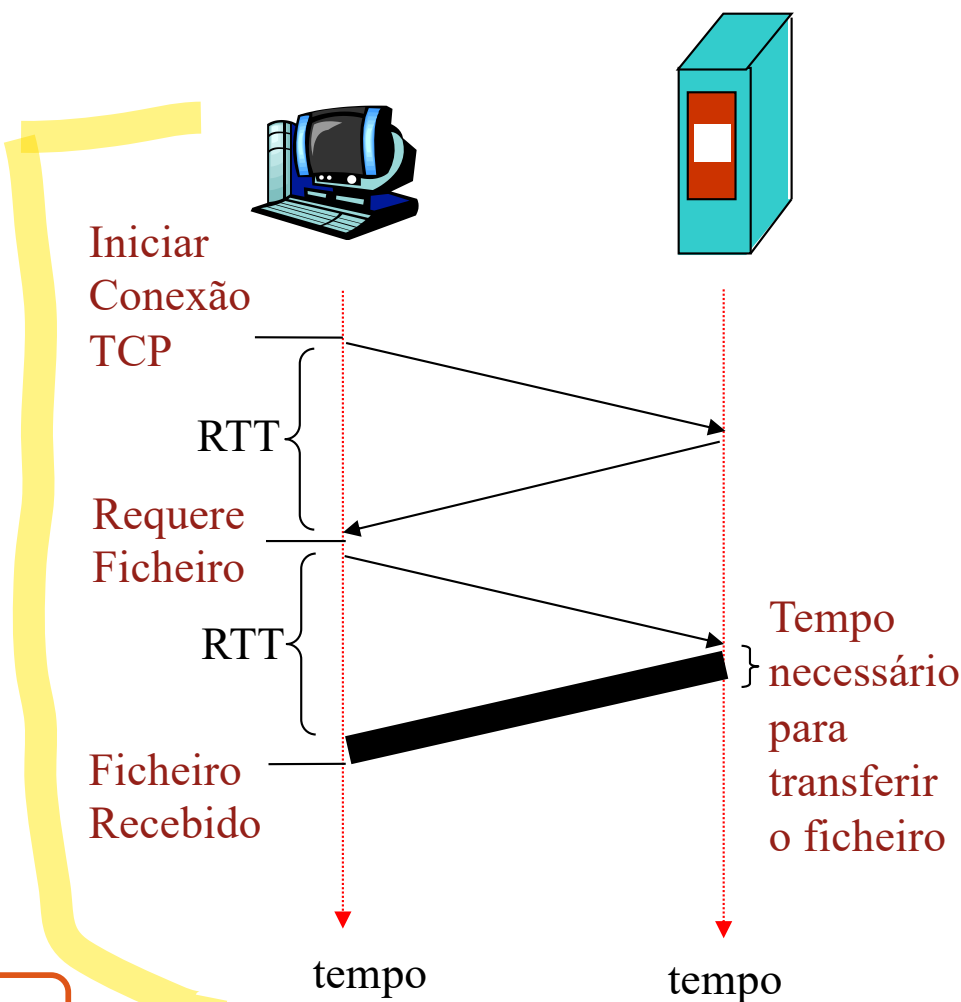


Definição de RTT: tempo que o sinal (1 bit) demora a ir do cliente para o servidor e voltar

Tempo de Resposta

- um RTT para iniciar uma conexão TCP
- um RTT para enviar a *request message* e receber o primeiro bit da *response message*
- tempo de transmissão do ficheiro

$$\text{total} = 2\text{RTT} + \text{tempo_transmissão}$$



Exercício



- **Pretende-se estimar o tempo mínimo necessário para obter um documento da Web. O documento é constituído por 6 objectos: o objecto base HTML e cinco imagens referenciadas no objecto base. O *browser* está ligado ao servidor HTTP por uma única linha com RTT de 20 ms. O tempo mínimo de transmissão na linha do objecto base HTML é de 8 ms e o tempo mínimo de transmissão na linha de cada imagem é de 80 ms. Admita que o *browser* só pode pedir as imagens quando receber completamente o objecto base. Admita que o utilizador o utilizador sabe o endereço IP do servidor, indicando-o no *browser*. A dimensão dos pacotes de estabelecimento de ligação, de confirmação de estabelecimento de ligação e de envio dos pedidos HTTP é desprezável. Os tempos de processamento dos pacotes são também desprezáveis. Não há mais tráfego nenhum na rede.**
- Ilustrando a situação com um diagrama temporal, qual o tempo necessário para obter o documento (todos os objectos) se utilizar HTTP não persistente com um máximo de 4 ligações paralelas?
- Ilustrando a situação com um diagrama temporal, qual o tempo necessário para obter o documento (todos os objectos) se utilizar HTTP/1.1 com *pipelining* em todos os pedidos?

Dados: $RTT = 20 \text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



Dados: $RTT = 20 \text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo

Dados: $RTT = 20 \text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

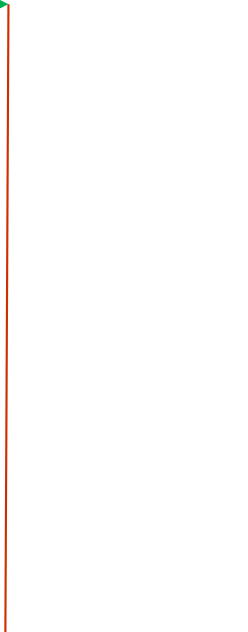
$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo

$t=0$ ----->



Dados: $RTT = 20 \text{ ms}$

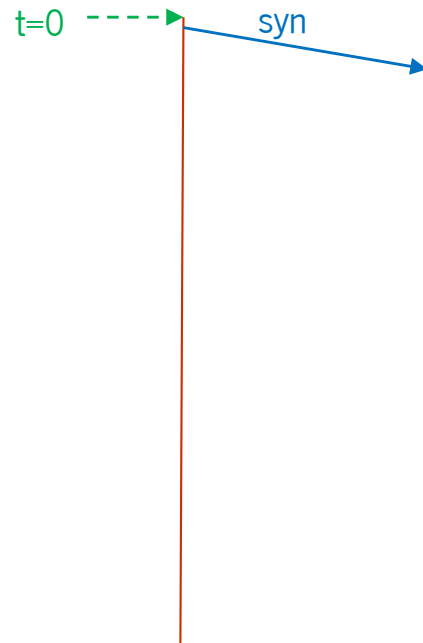
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20 \text{ ms}$

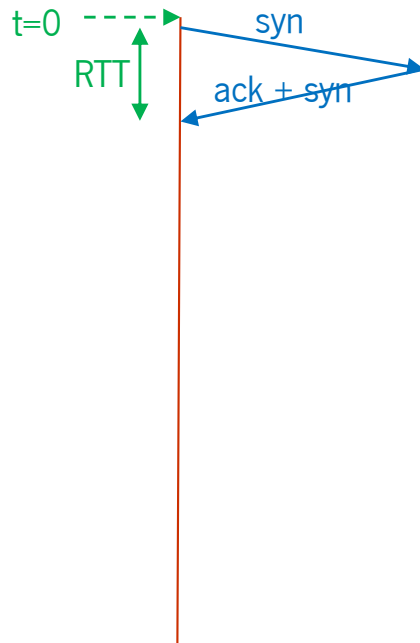
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20\text{ ms}$

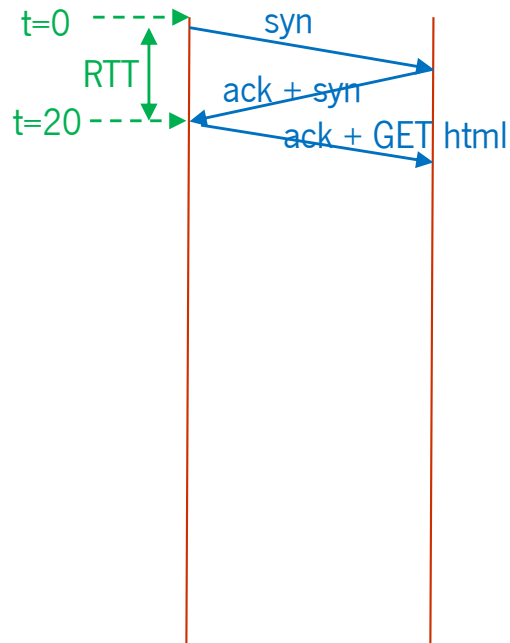
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20 \text{ ms}$

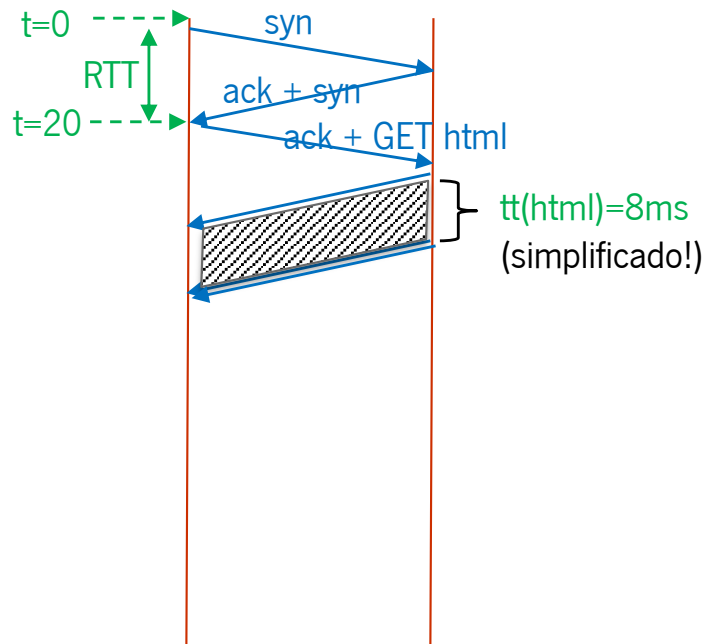
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20\text{ ms}$

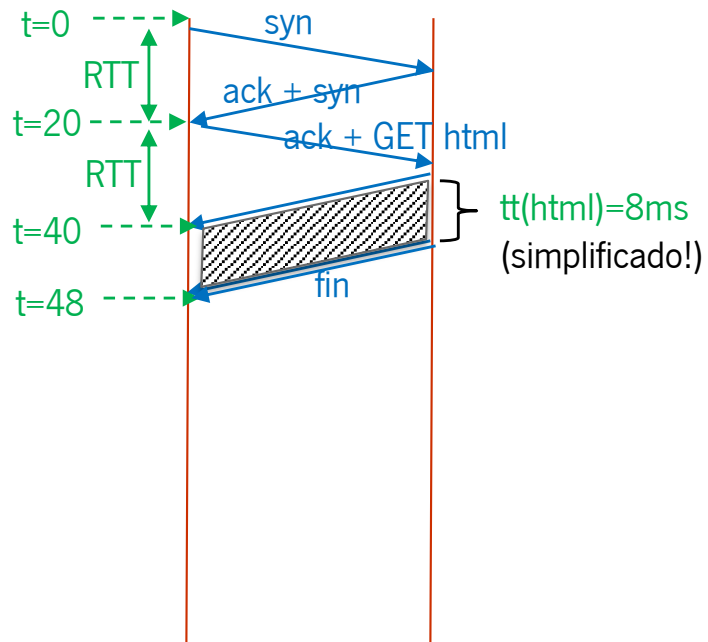
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20 \text{ ms}$

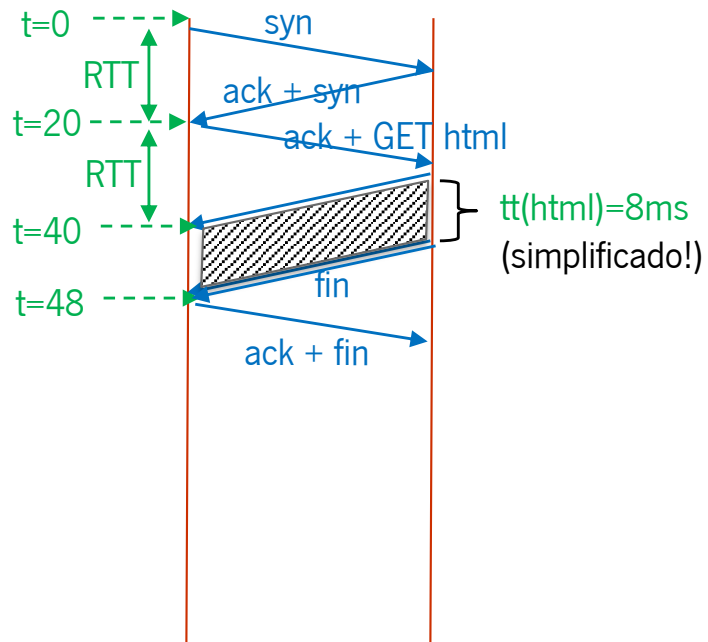
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20 \text{ ms}$

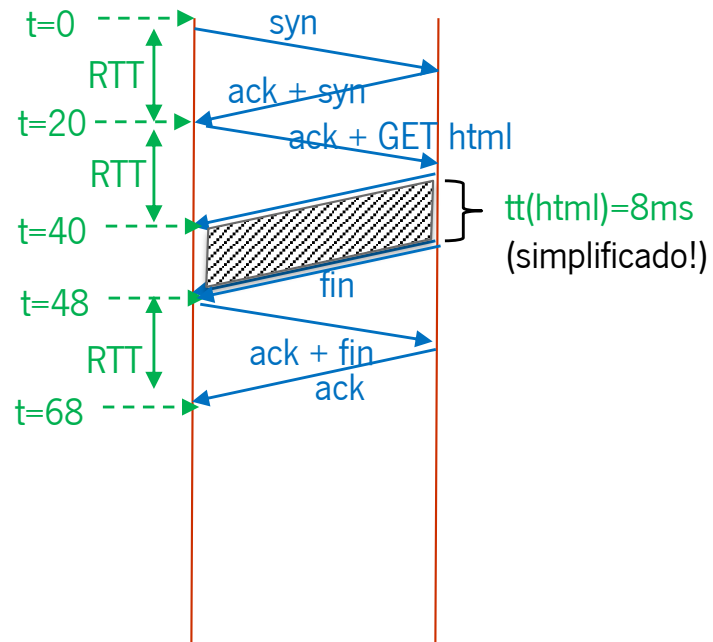
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20 \text{ ms}$

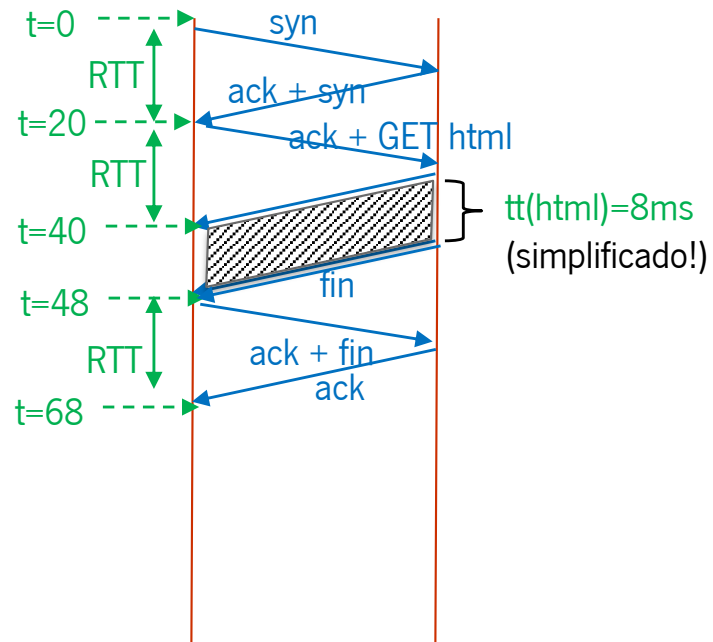
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20\text{ ms}$

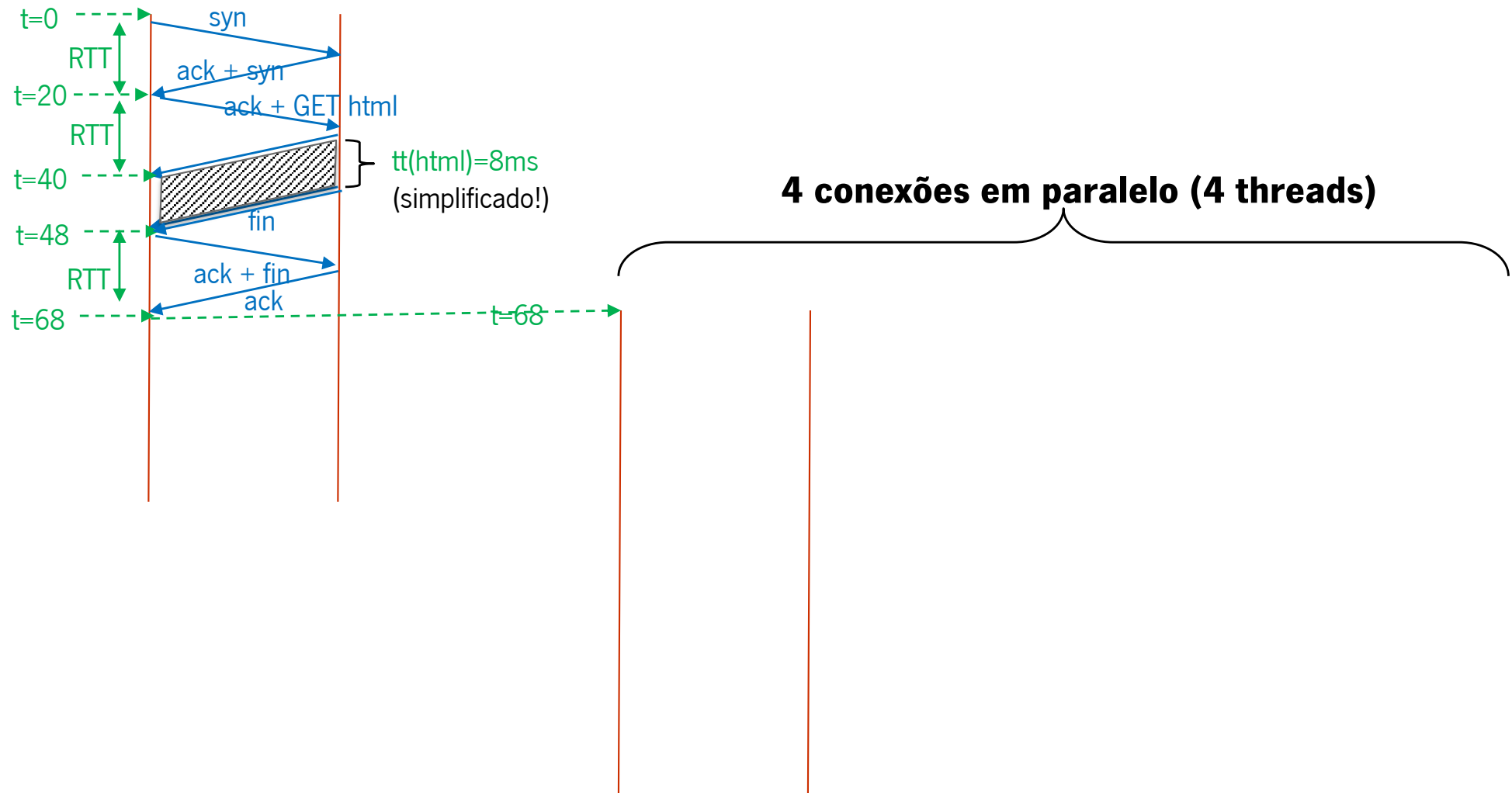
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: RTT = 20 ms

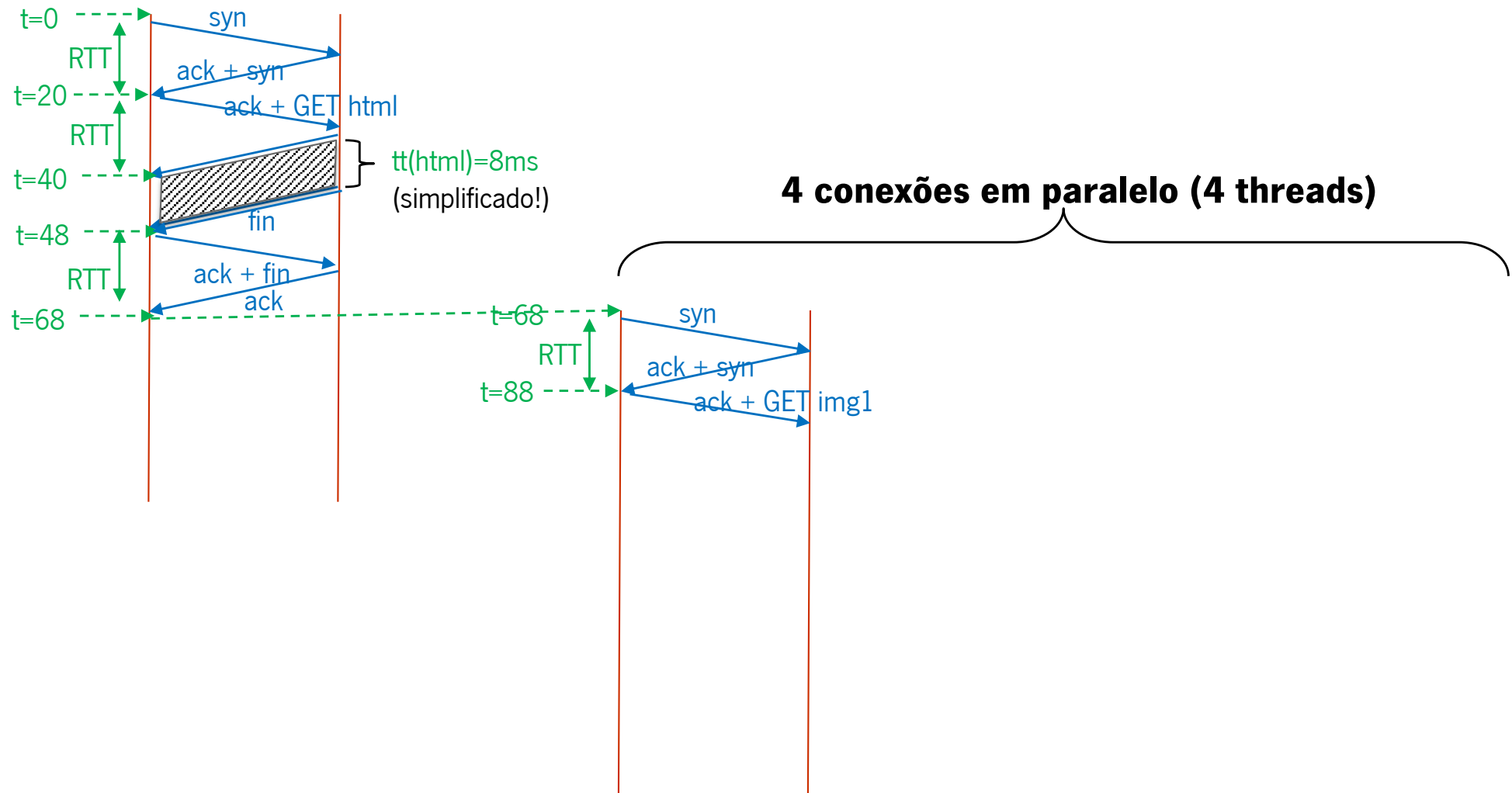
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20\text{ ms}$

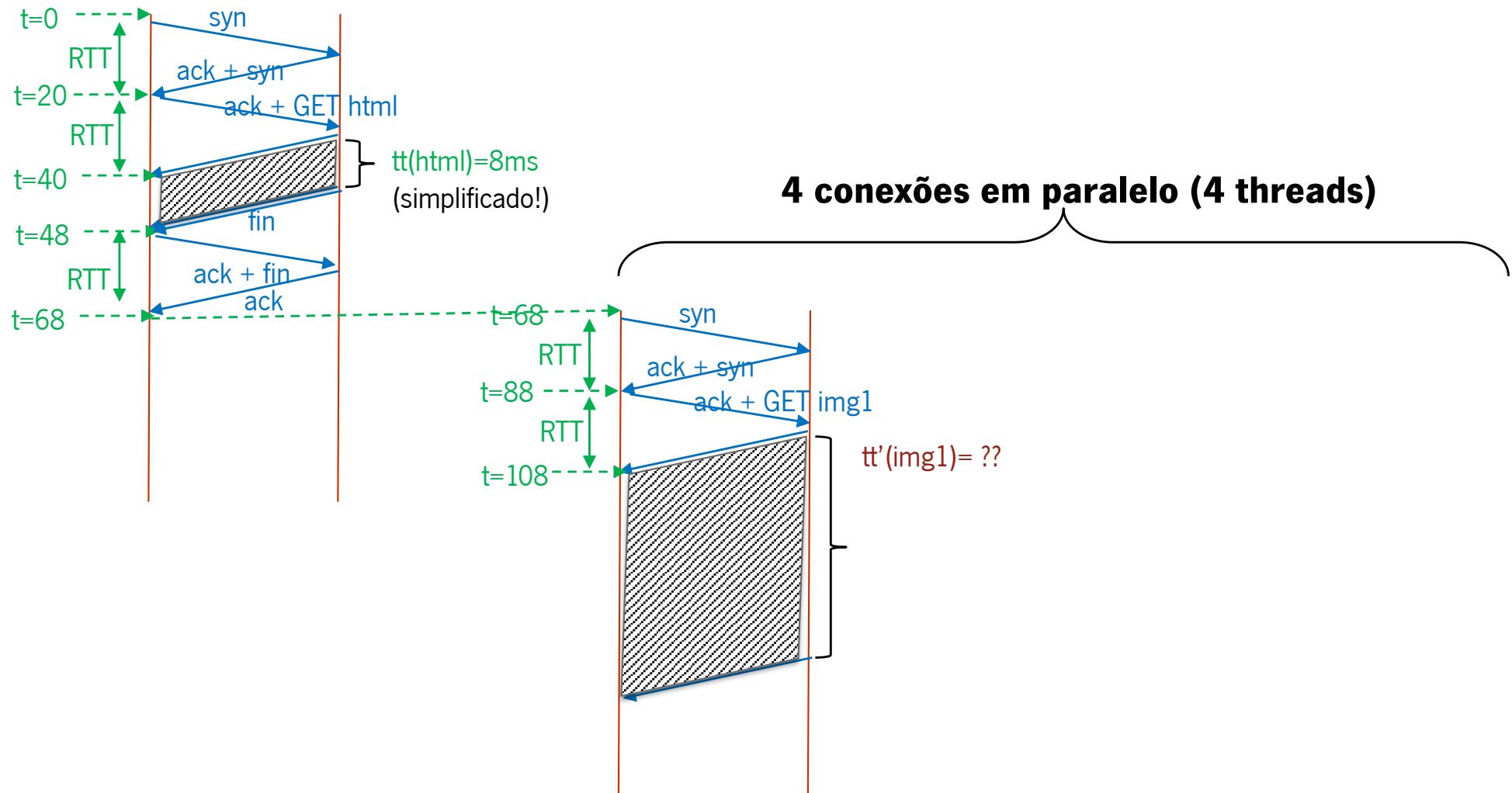
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20\text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

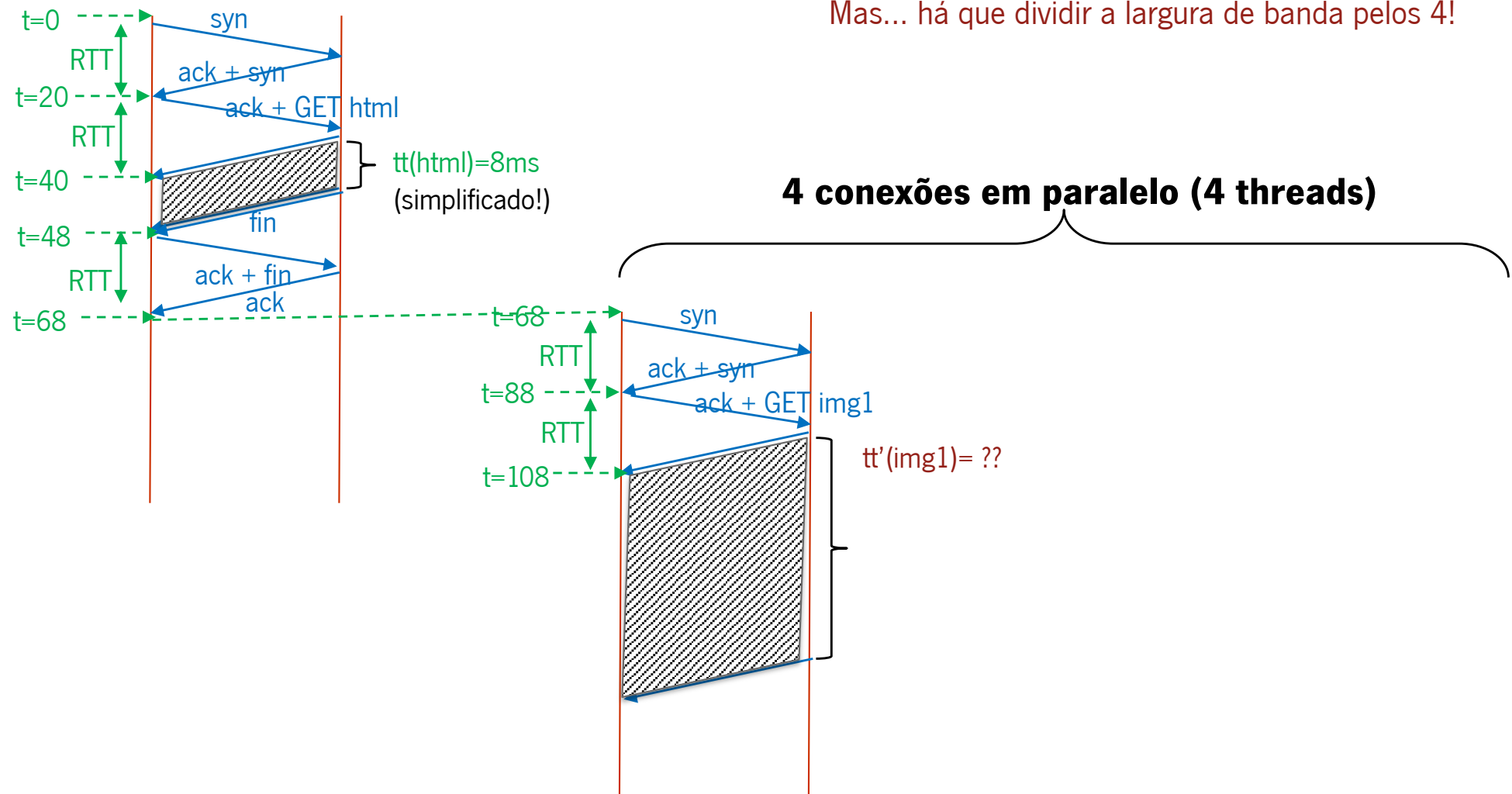
$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo

Mas... há que dividir a largura de banda pelos 4!



Dados: $RTT = 20\text{ ms}$

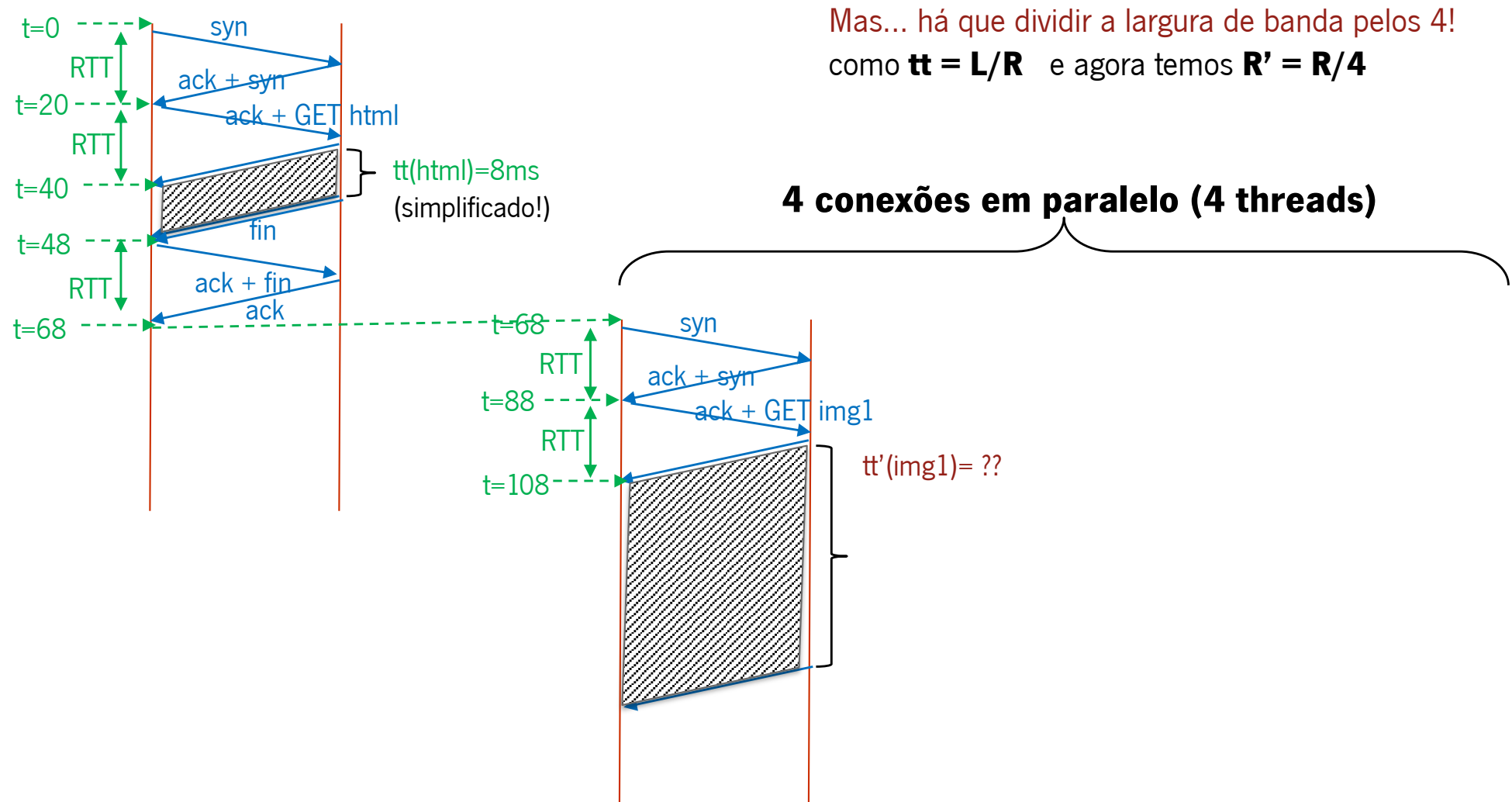
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Dados: $RTT = 20 \text{ ms}$

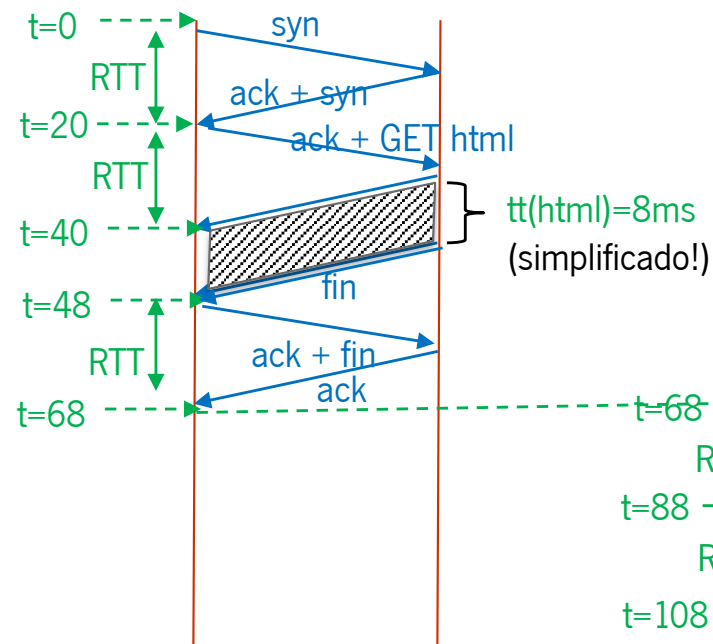
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$

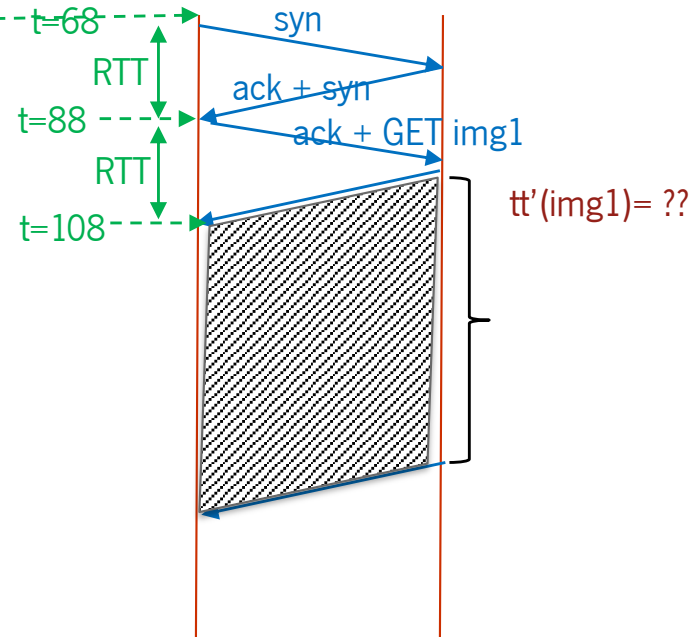


a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Mas... há que dividir a largura de banda pelos 4!
como $tt = L/R$ e agora temos $R' = R/4$
então $tt' = L/R' = L/(R/4) = 4 * L/R \Rightarrow tt' = 4 tt$

4 conexões em paralelo (4 threads)



Dados: RTT = 20 ms

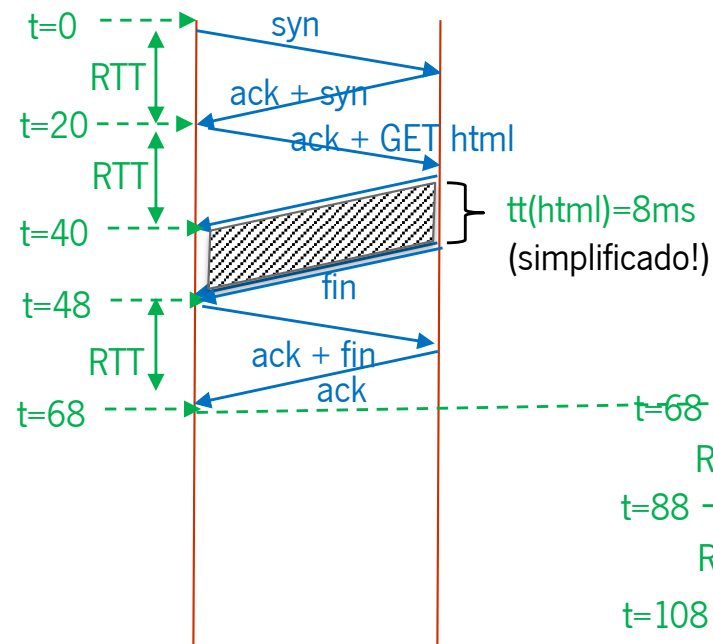
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms

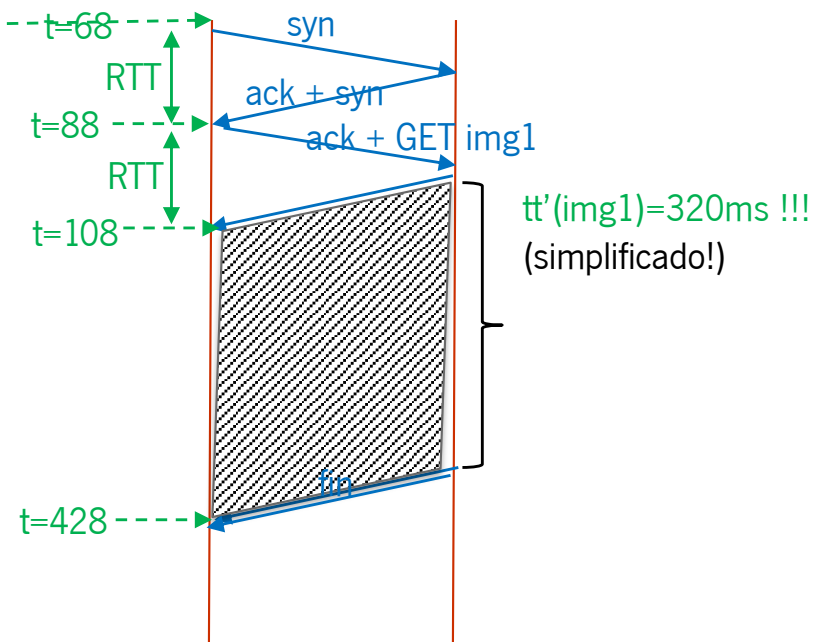


a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Mas... há que dividir a largura de banda pelos 4!
como $tt = L/R$ e agora temos $R' = R/4$
então $tt' = L/R' = L/(R/4) = 4 * L/R \Rightarrow tt' = 4 tt$

4 conexões em paralelo (4 threads)



Dados: RTT = 20 ms

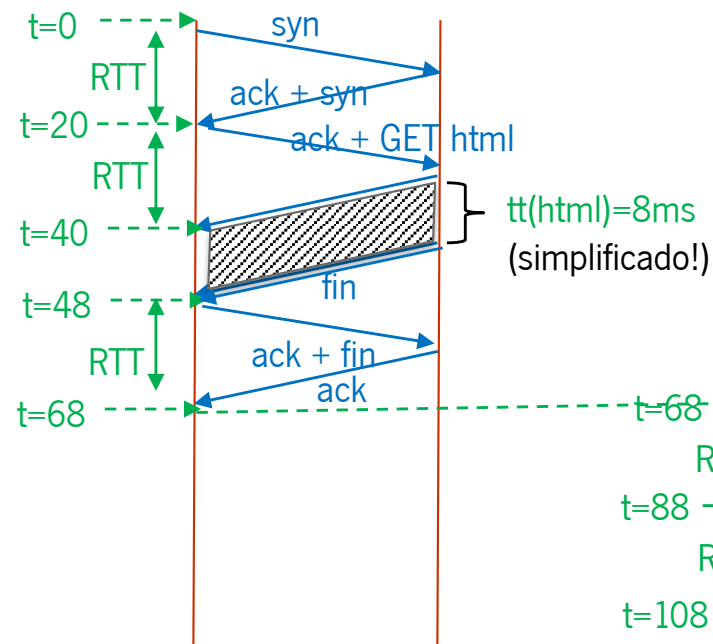
6 Objectos $\rightarrow$ { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms

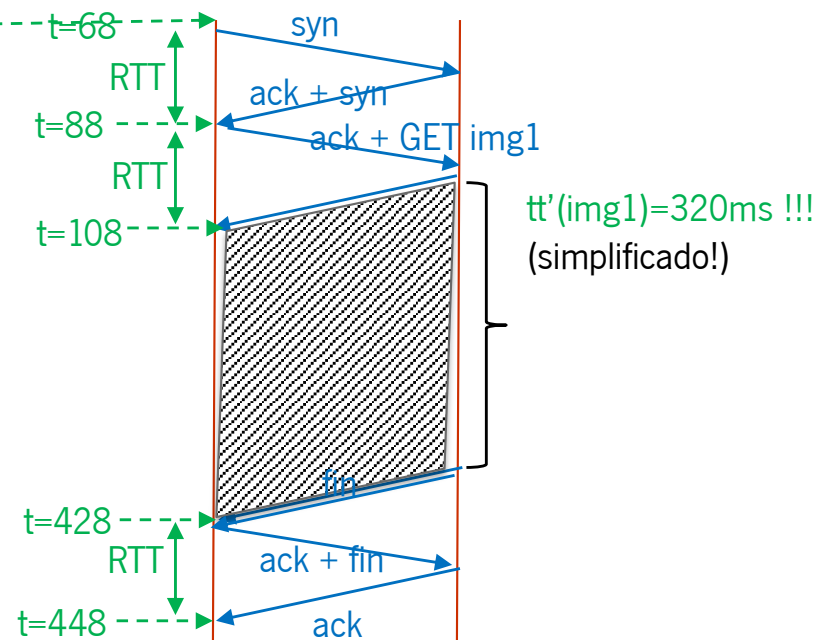


a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Mas... há que dividir a largura de banda pelos 4!
como $tt = L/R$ e agora temos $R' = R/4$
então $tt' = L/R' = L/(R/4) = 4 * L/R \Rightarrow tt' = 4 tt$

4 conexões em paralelo (4 threads)



Dados: RTT = 20 ms

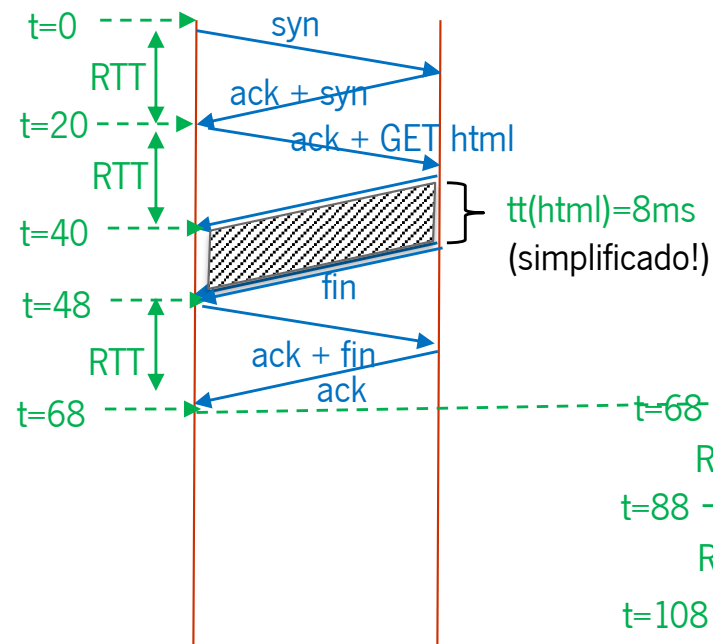
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms

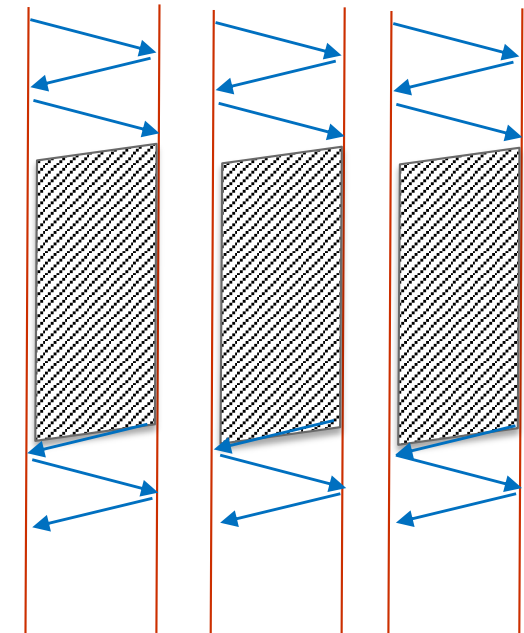
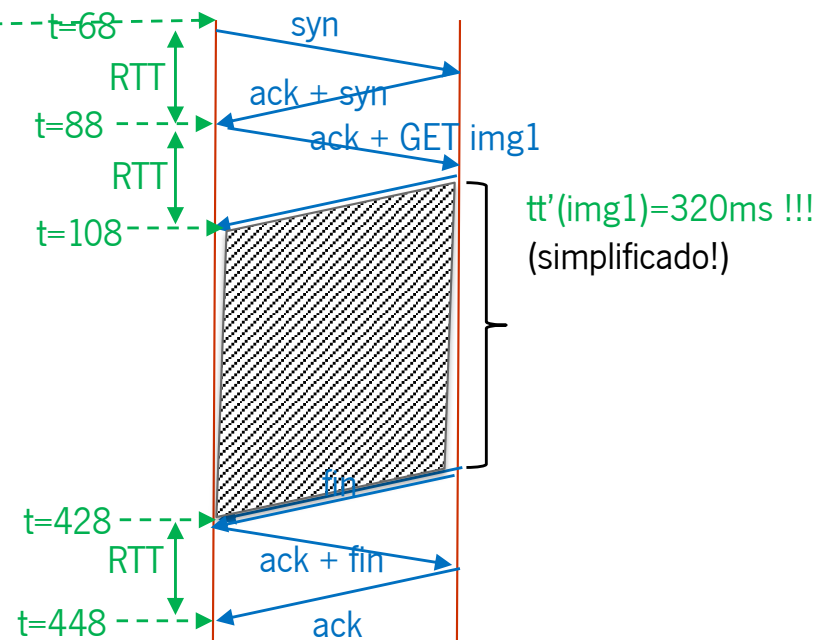


a) HTTP/1.0 Não persistente, com 4 conexões em paralelo



Mas... há que dividir a largura de banda pelos 4!
como $tt = L/R$ e agora temos $R' = R/4$
então $tt' = L/R' = L/(R/4) = 4 * L/R \Rightarrow tt' = 4 tt$

4 conexões em paralelo (4 threads)



Dados: $RTT = 20 \text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) ... continuação: descarregar a ultima imagem numa nova conexão

Dados: $RTT = 20 \text{ ms}$

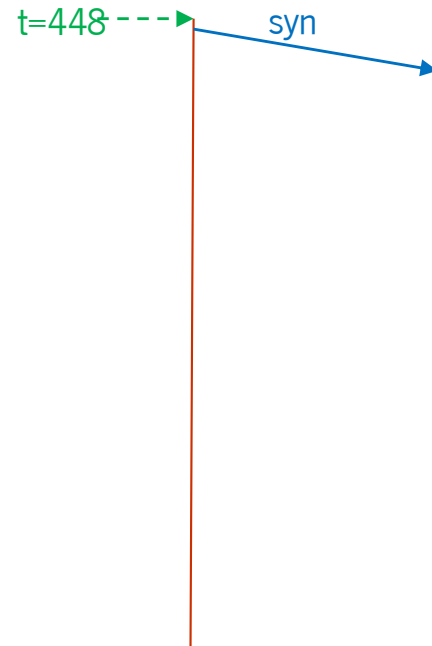
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) ... continuação: descarregar a ultima imagem numa nova conexão



Dados: $RTT = 20 \text{ ms}$

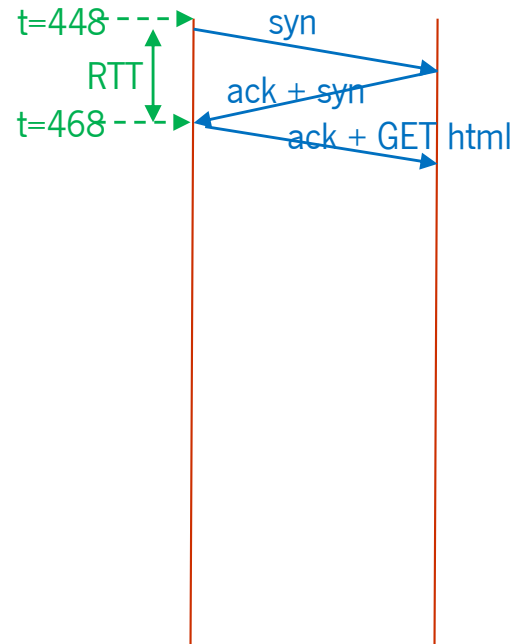
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) ... continuação: descarregar a ultima imagem numa nova conexão



Dados: $RTT = 20 \text{ ms}$

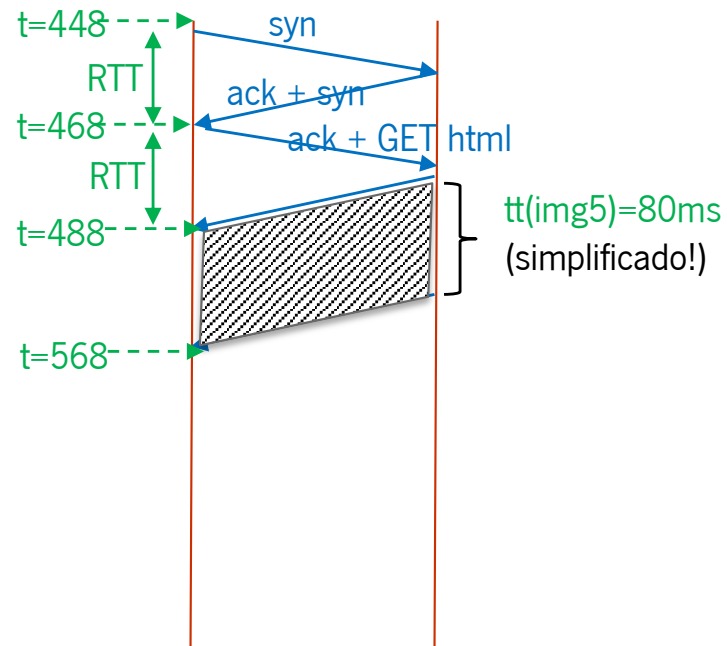
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) ... continuação: descarregar a ultima imagem numa nova conexão



Dados: $RTT = 20 \text{ ms}$

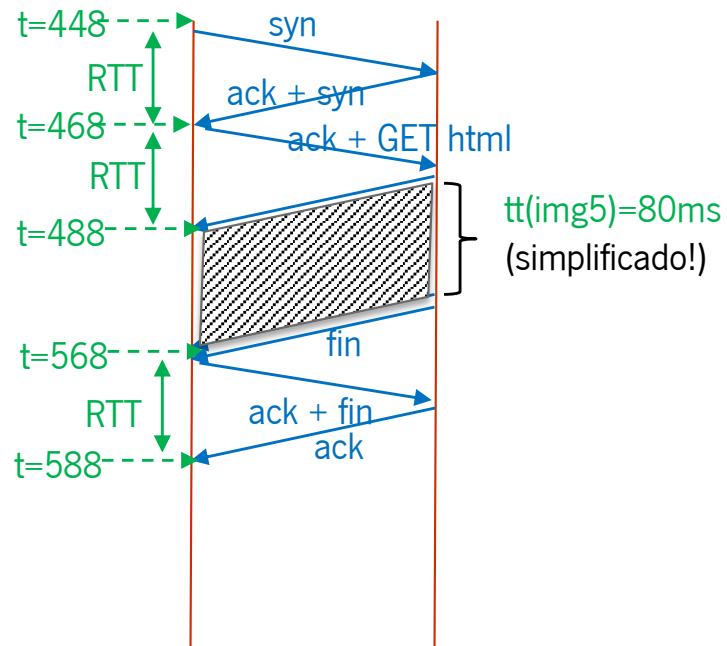
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) ... continuação: descarregar a ultima imagem numa nova conexão



Dados: $RTT = 20 \text{ ms}$

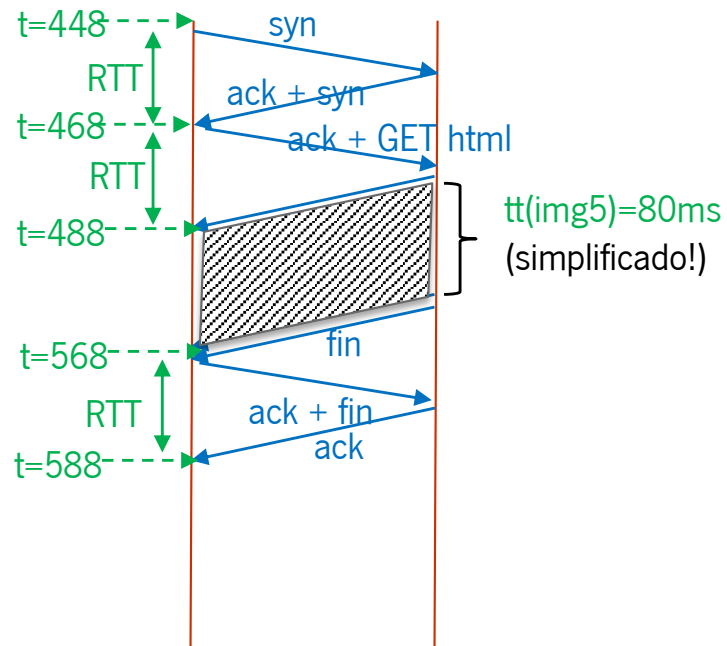
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



a) ... continuação: descarregar a ultima imagem numa nova conexão



IMG5

T.a.total = 568 ms (ou 588 ms contando com fecho conexão)

Dados: $RTT = 20 \text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo

Dados: $RTT = 20\text{ ms}$

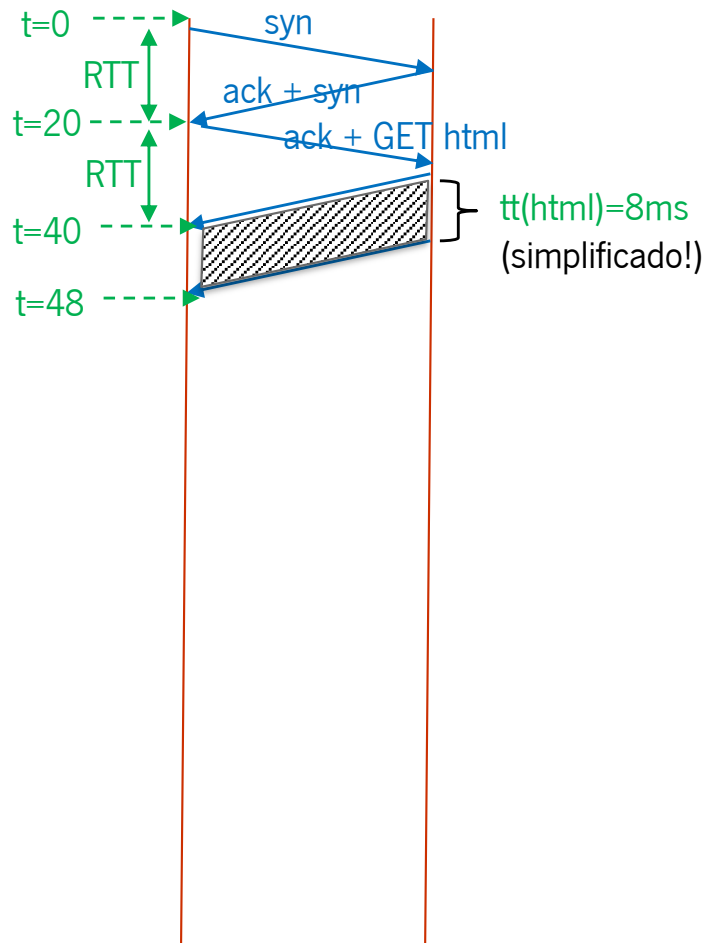
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: $RTT = 20\text{ ms}$

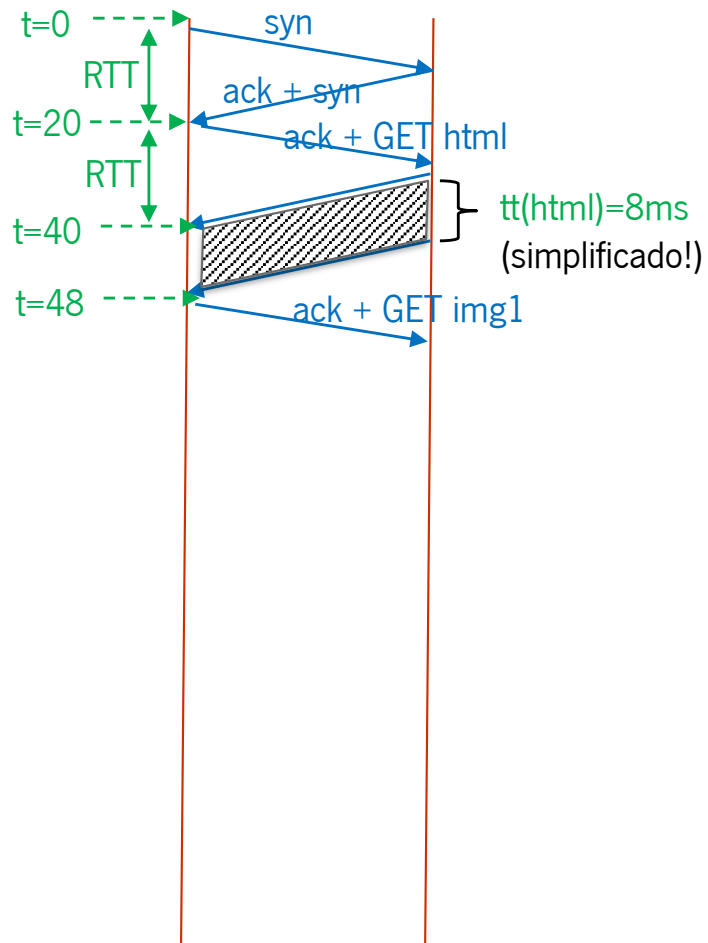
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: $RTT = 20\text{ ms}$

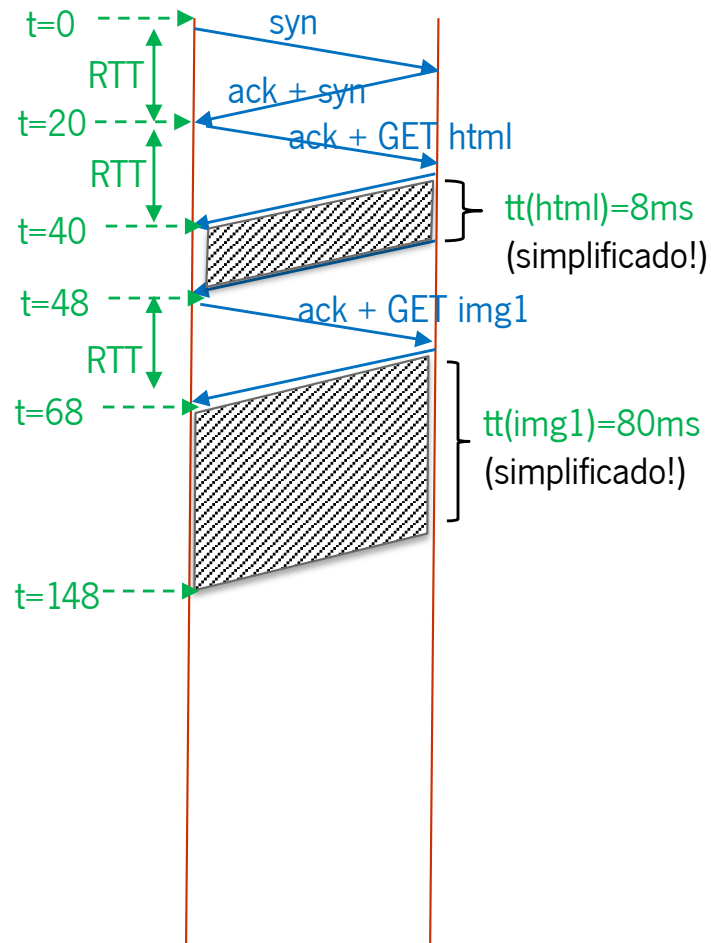
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: $RTT = 20\text{ ms}$

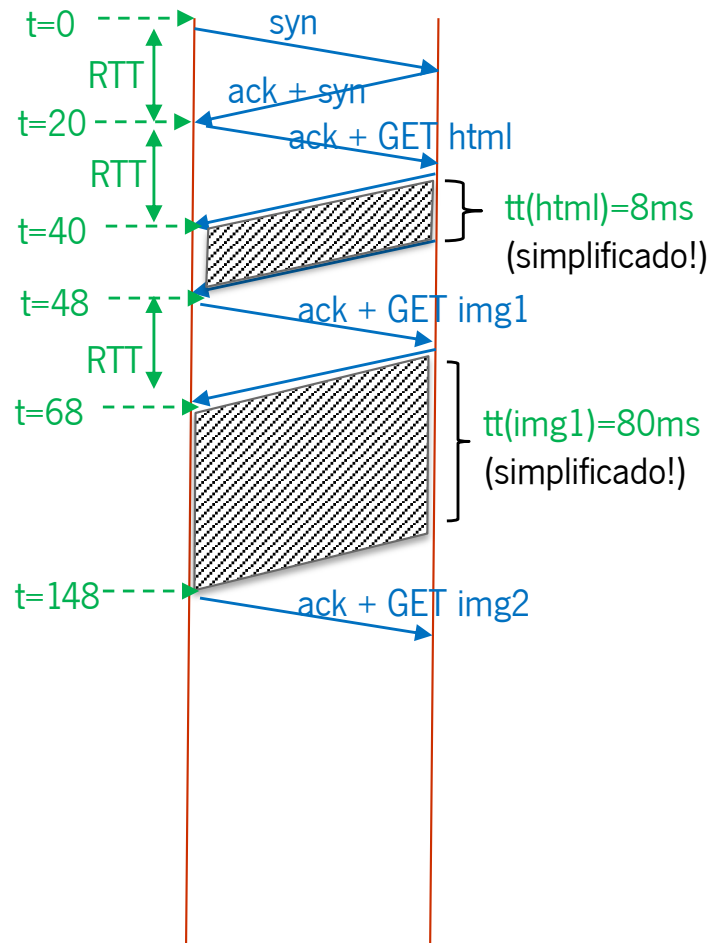
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: $RTT = 20\text{ ms}$

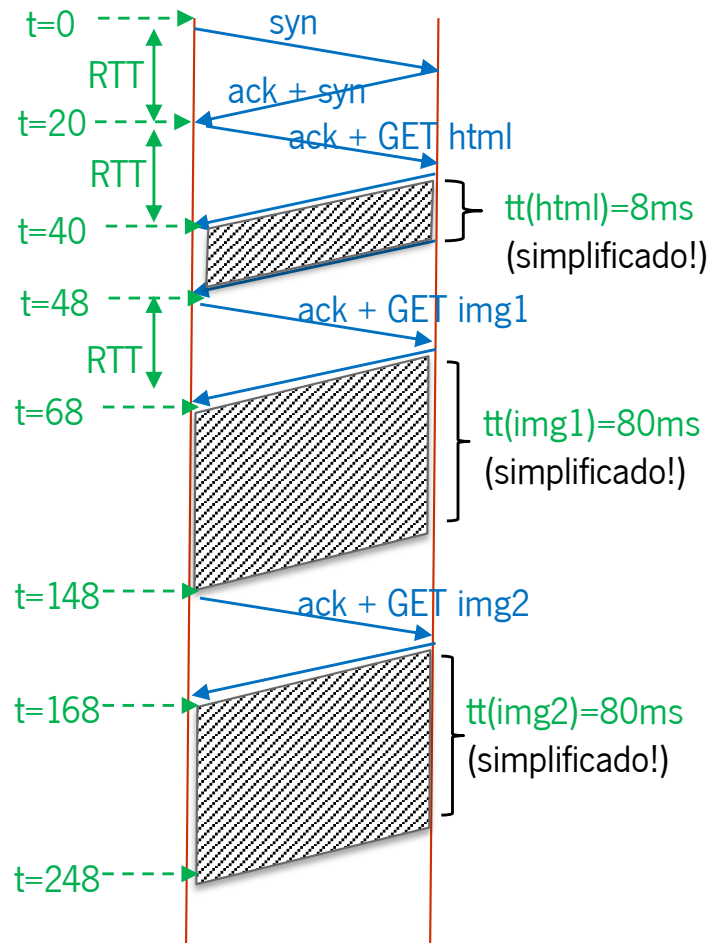
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

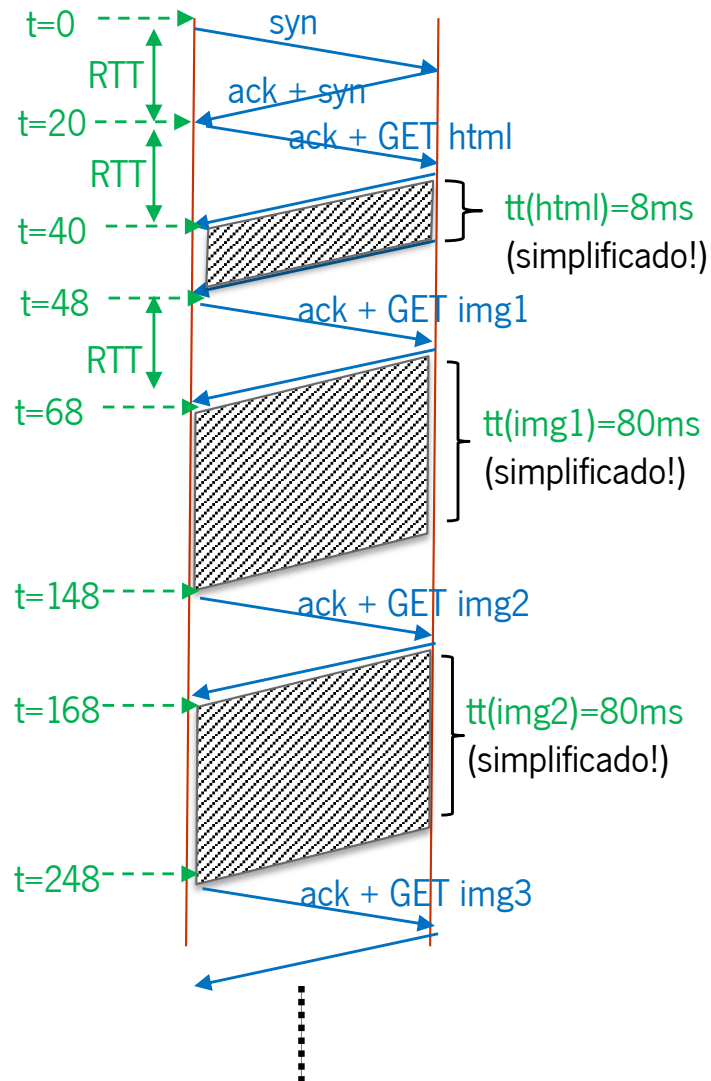
6 Objectos $\rightarrow$ { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

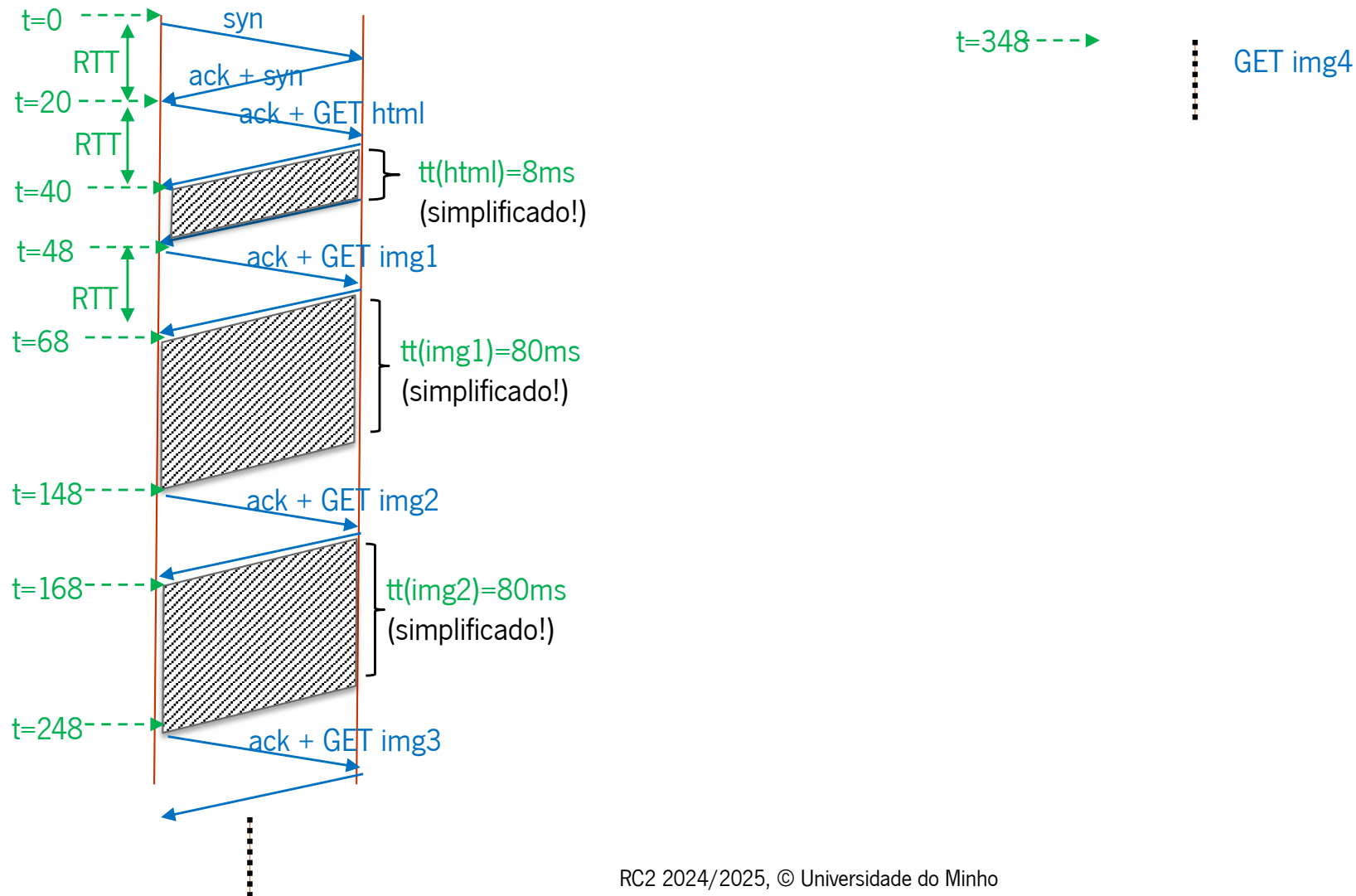
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

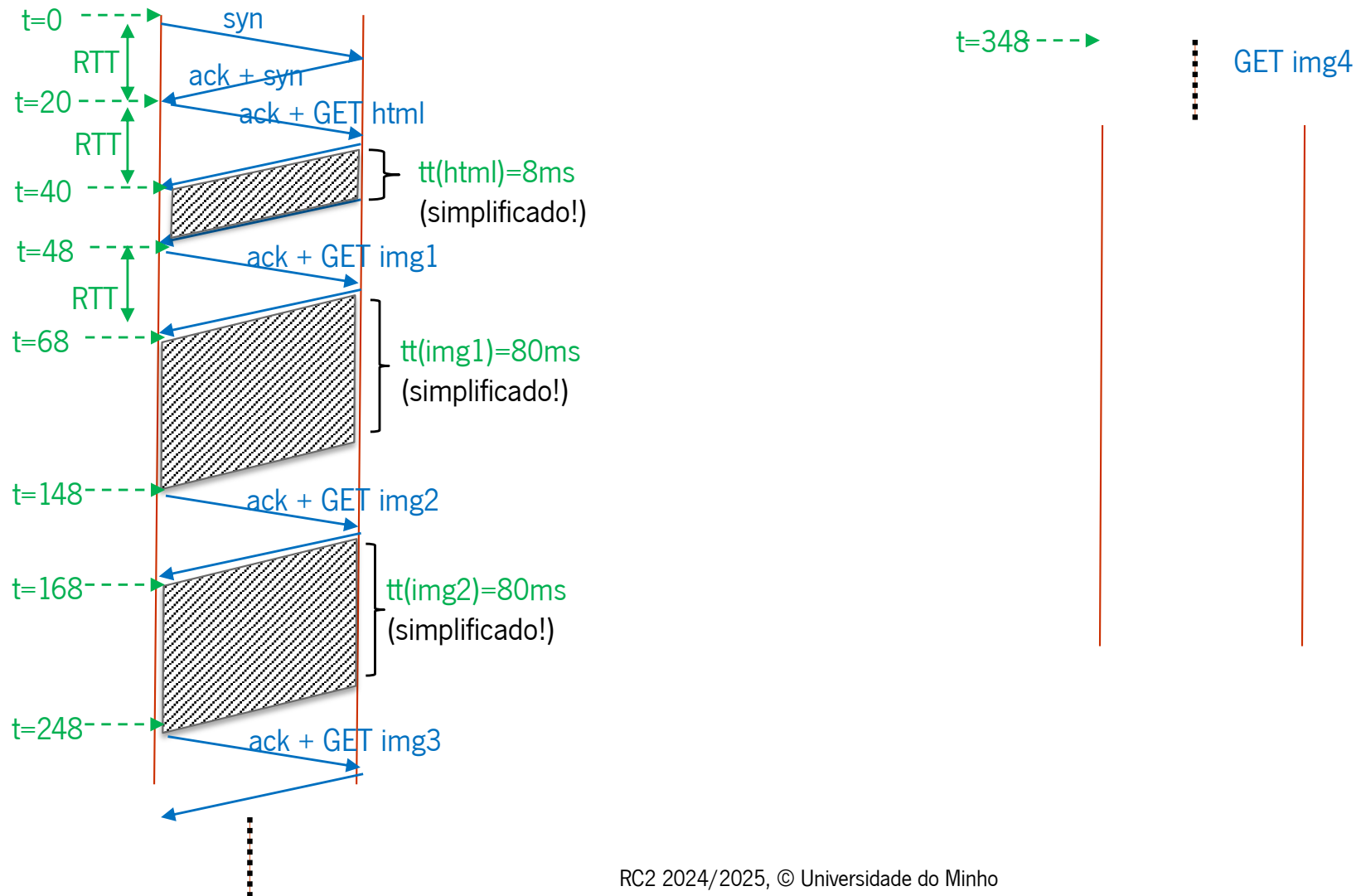
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

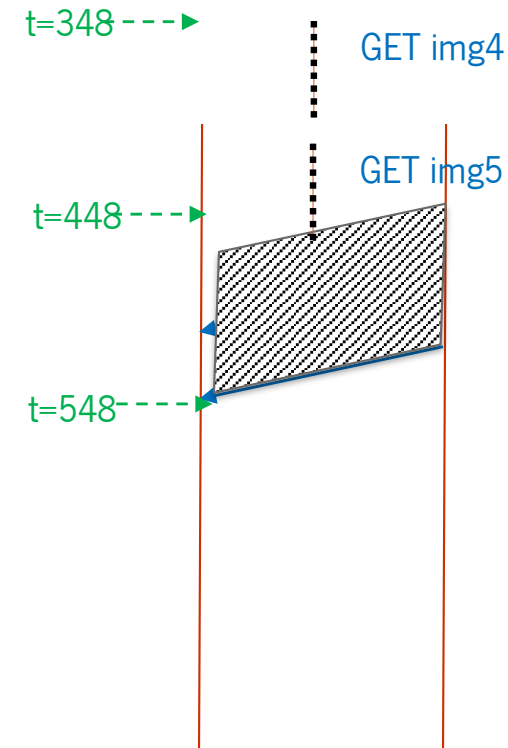
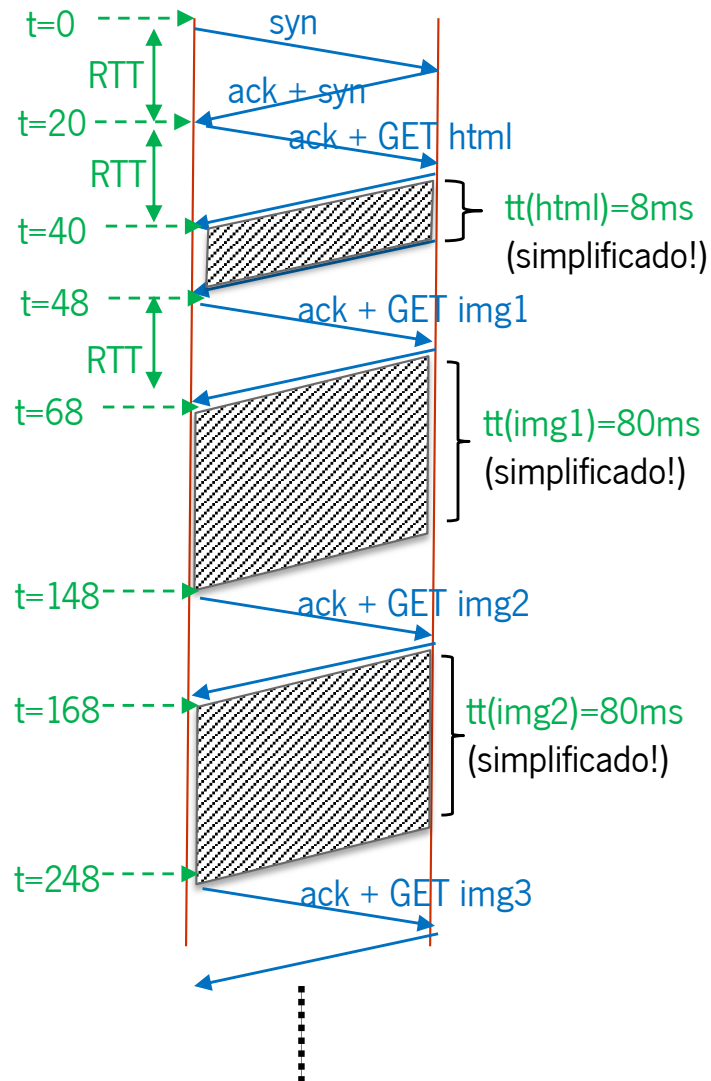
6 Objectos $\rightarrow$ { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

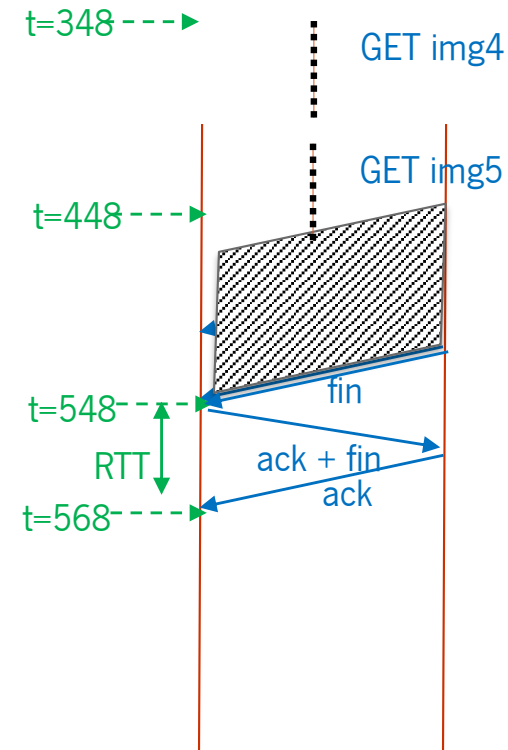
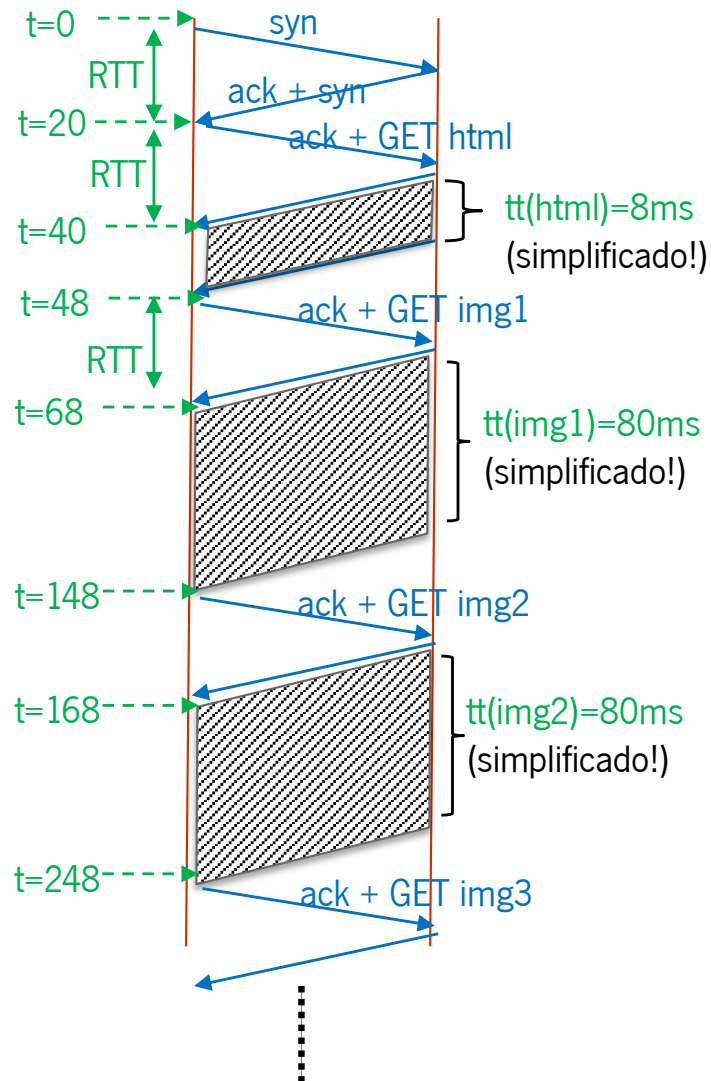
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

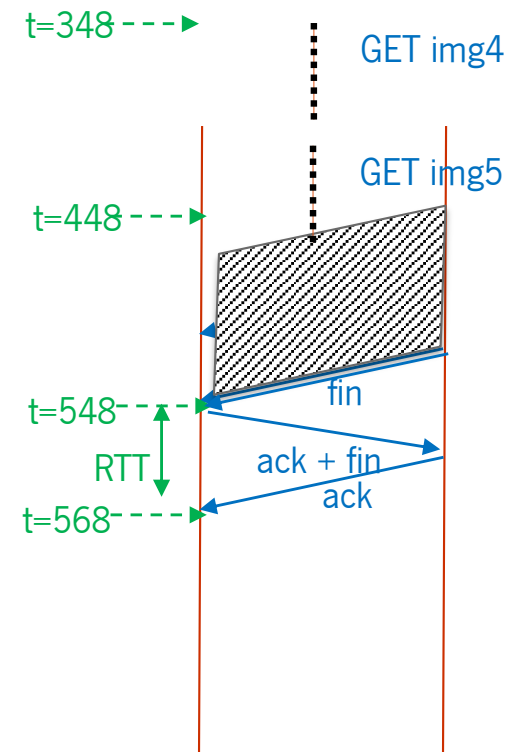
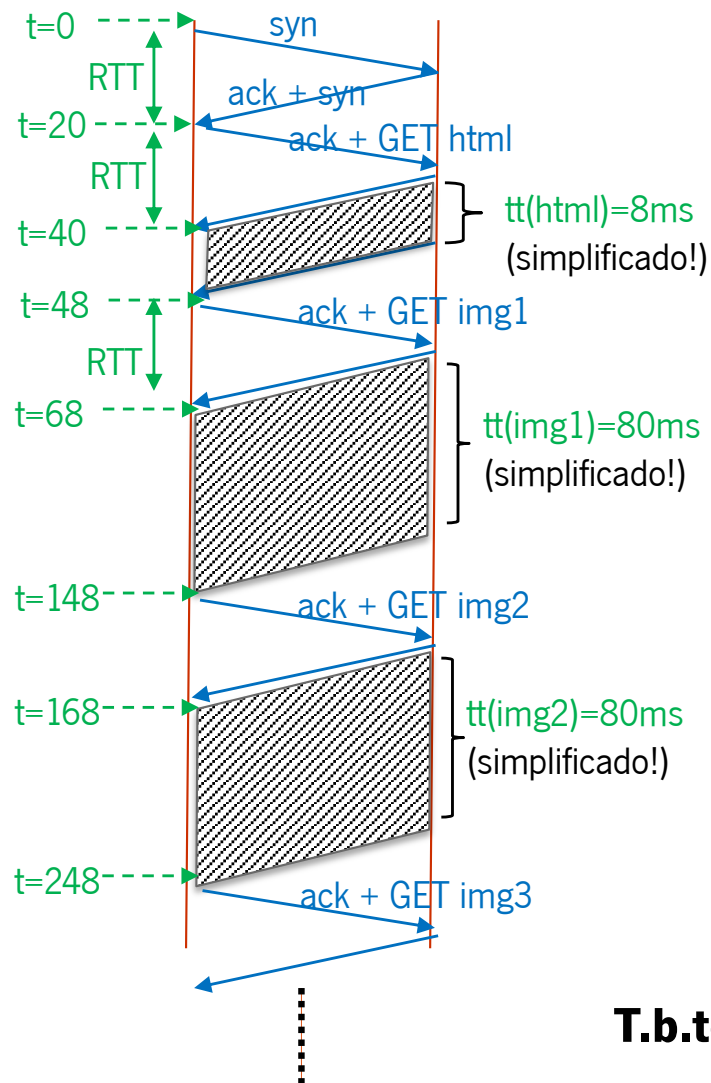
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



b) HTTP/1.1 persistente, sem pipeline, sem conexões em paralelo



T.b.total = 548 ms (ou 568 ms contando com fecho conexão)

Dados: $RTT = 20 \text{ ms}$

6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8 \text{ ms}$

$tt(\text{img}) = 80 \text{ ms}$



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo

Dados: $RTT = 20\text{ ms}$

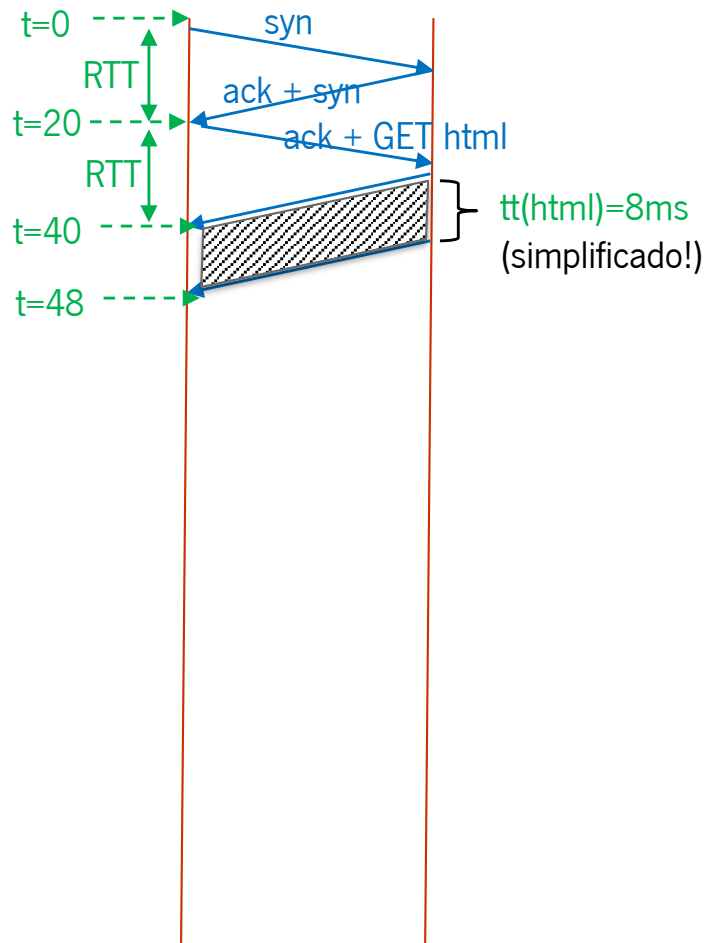
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

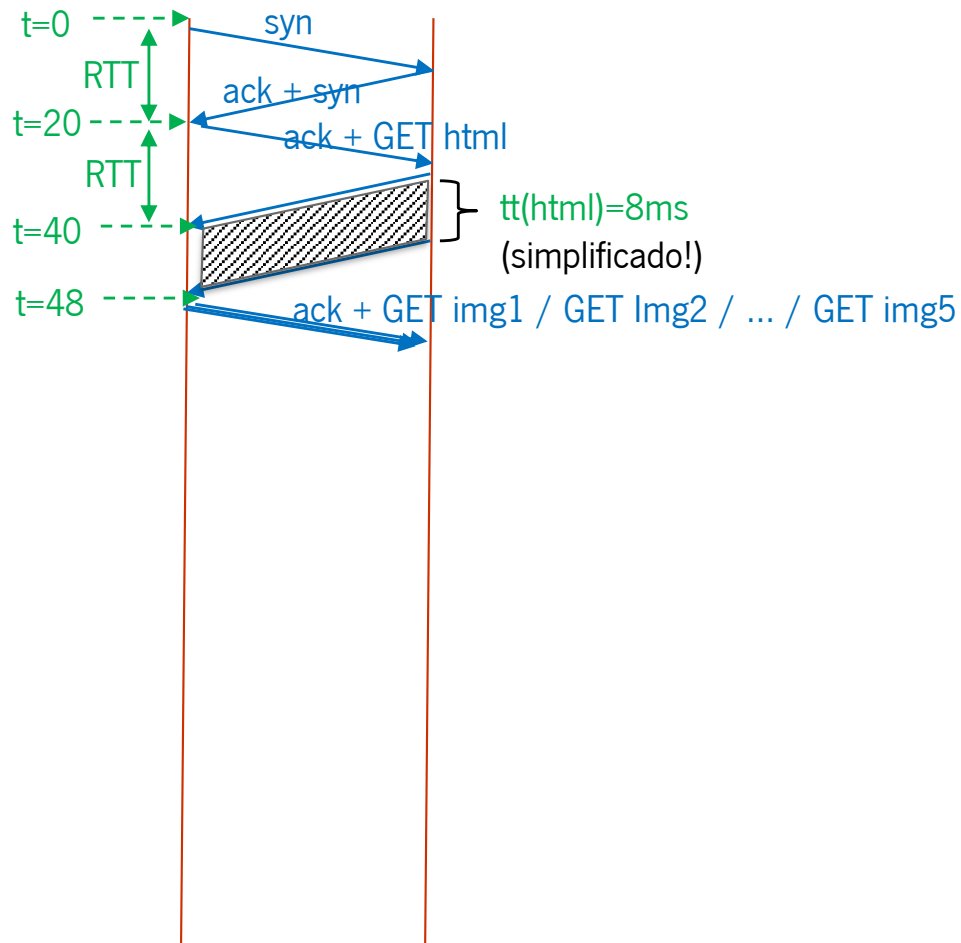
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo



Dados: $RTT = 20\text{ ms}$

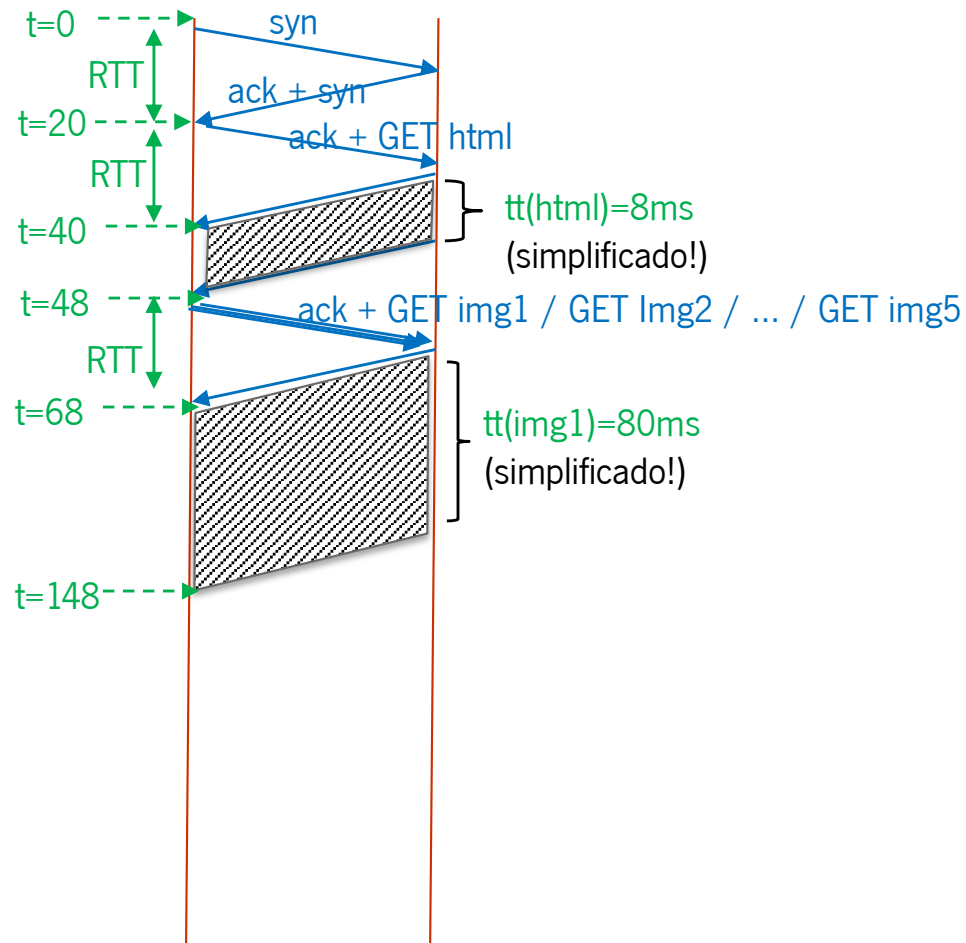
6 Objectos $\rightarrow \{ \text{html}, \text{img1}, \text{img2}, \text{img3}, \text{img4}, \text{img5} \}$

$tt(\text{html}) = 8\text{ ms}$

$tt(\text{img}) = 80\text{ ms}$



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

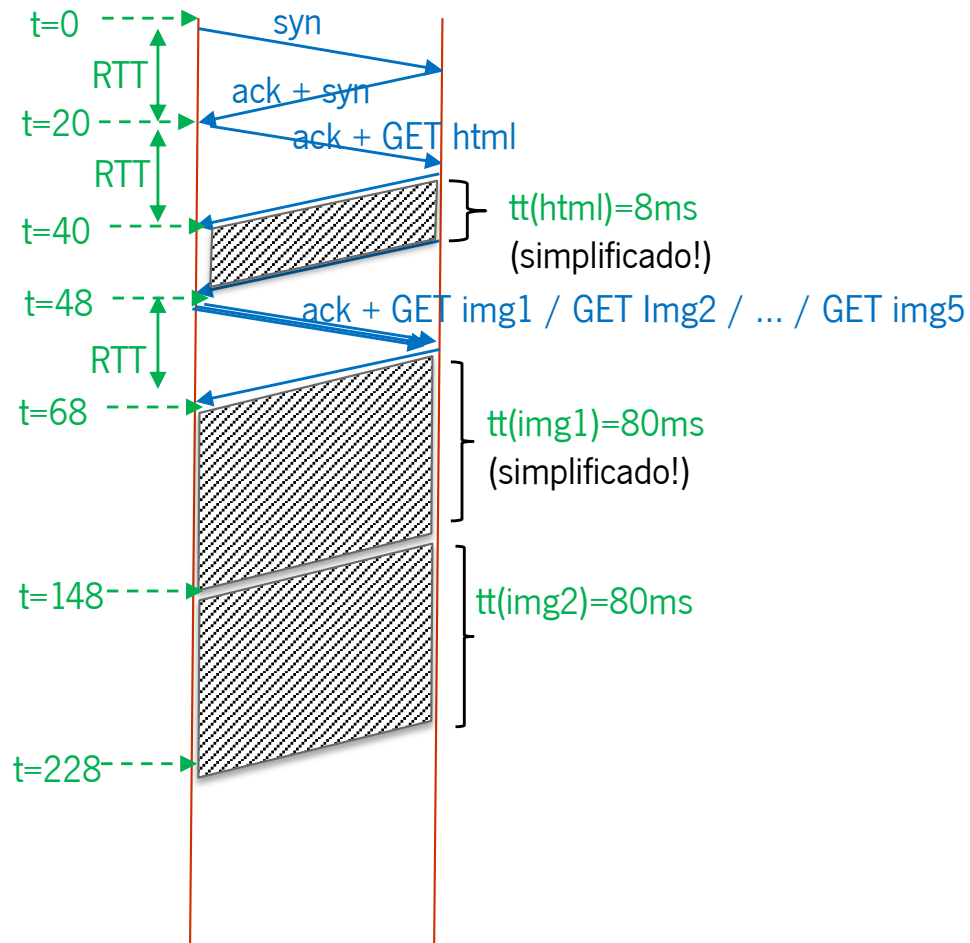
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

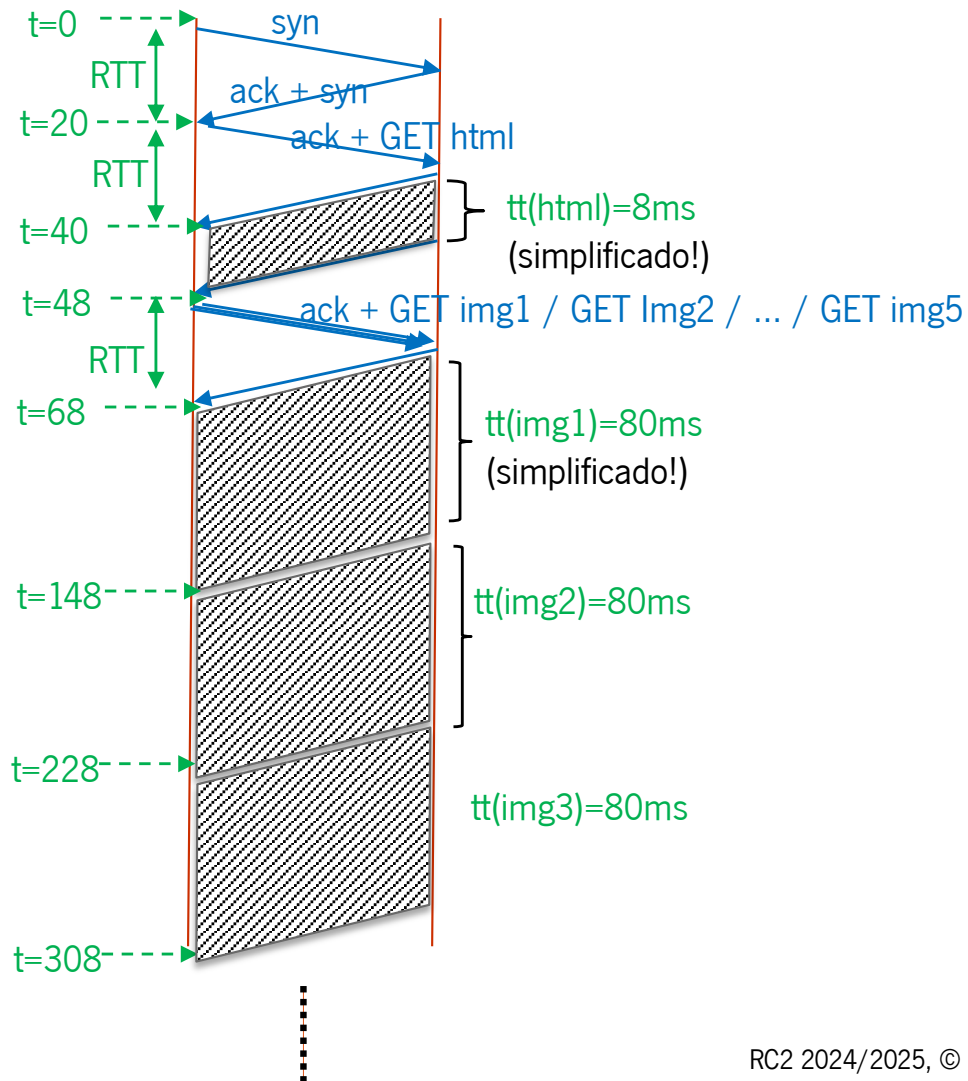
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt(html) = 8 ms

tt(img) = 80 ms



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

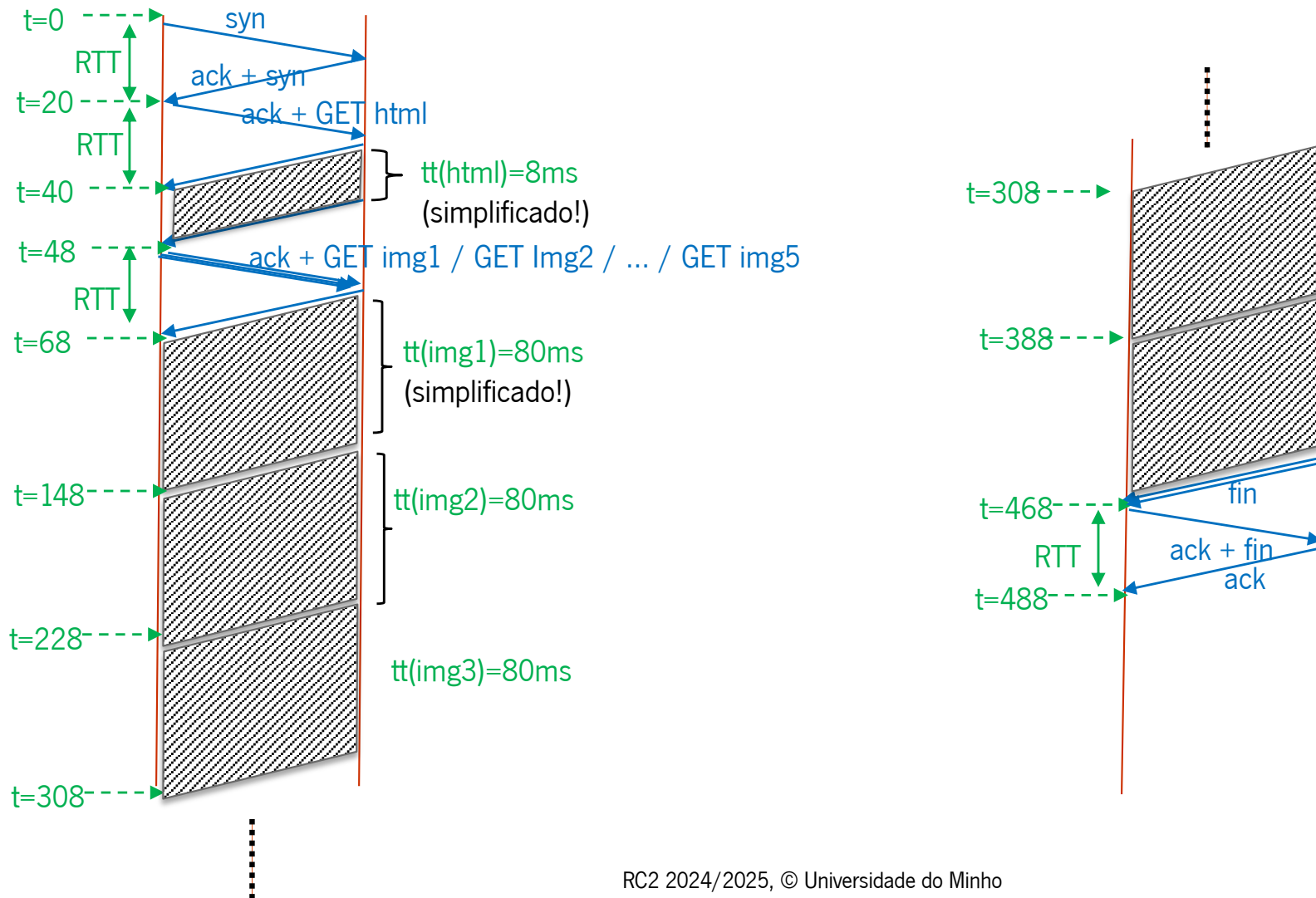
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt(html) = 8 ms

tt(img) = 80 ms



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo



Dados: RTT = 20 ms

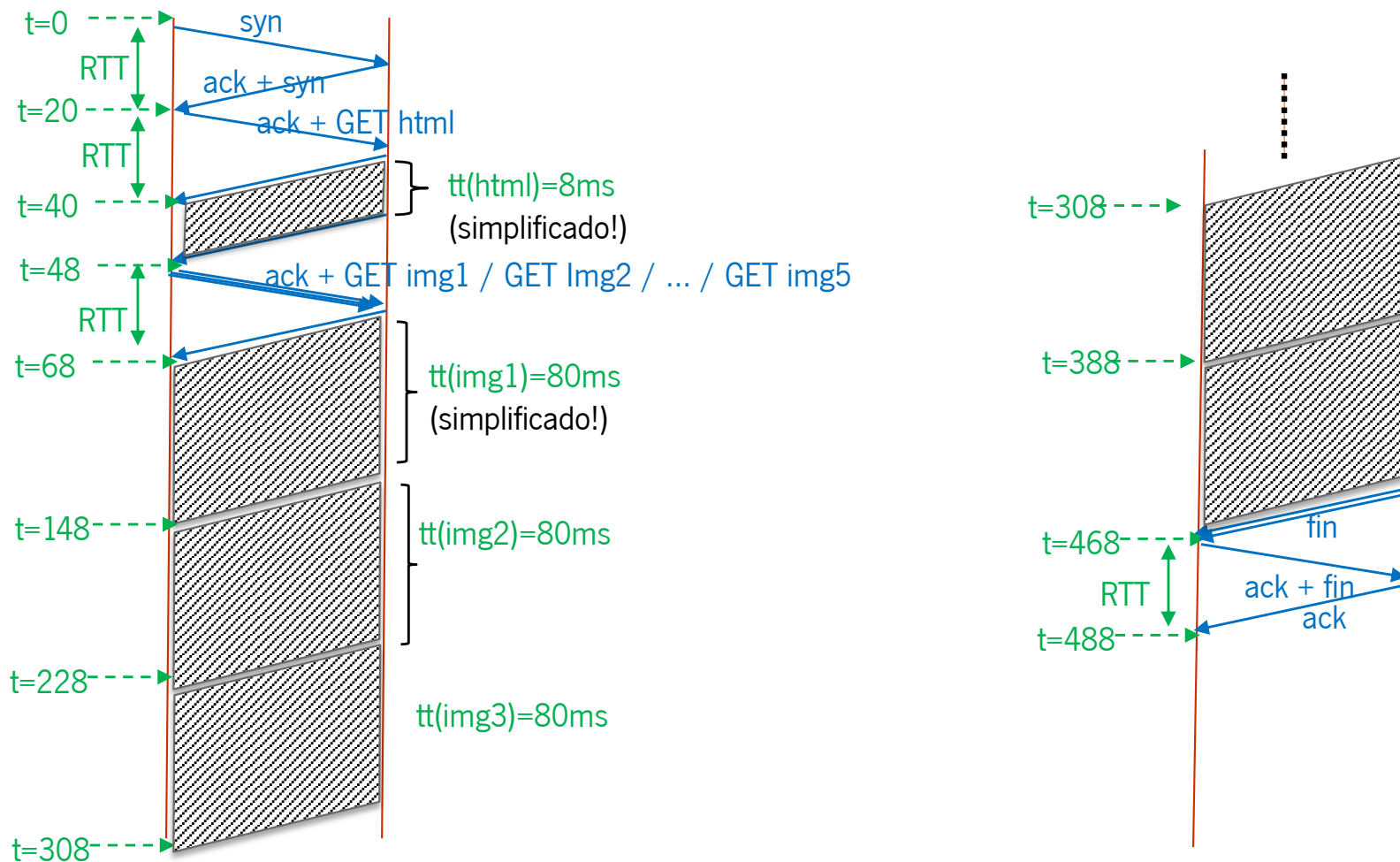
6 Objectos → { html, img1, img2, img3, img4, img5 }

tt (html) = 8 ms

tt(img) = 80 ms



c) HTTP/1.1 persistente, com pipeline, sem conexões em paralelo

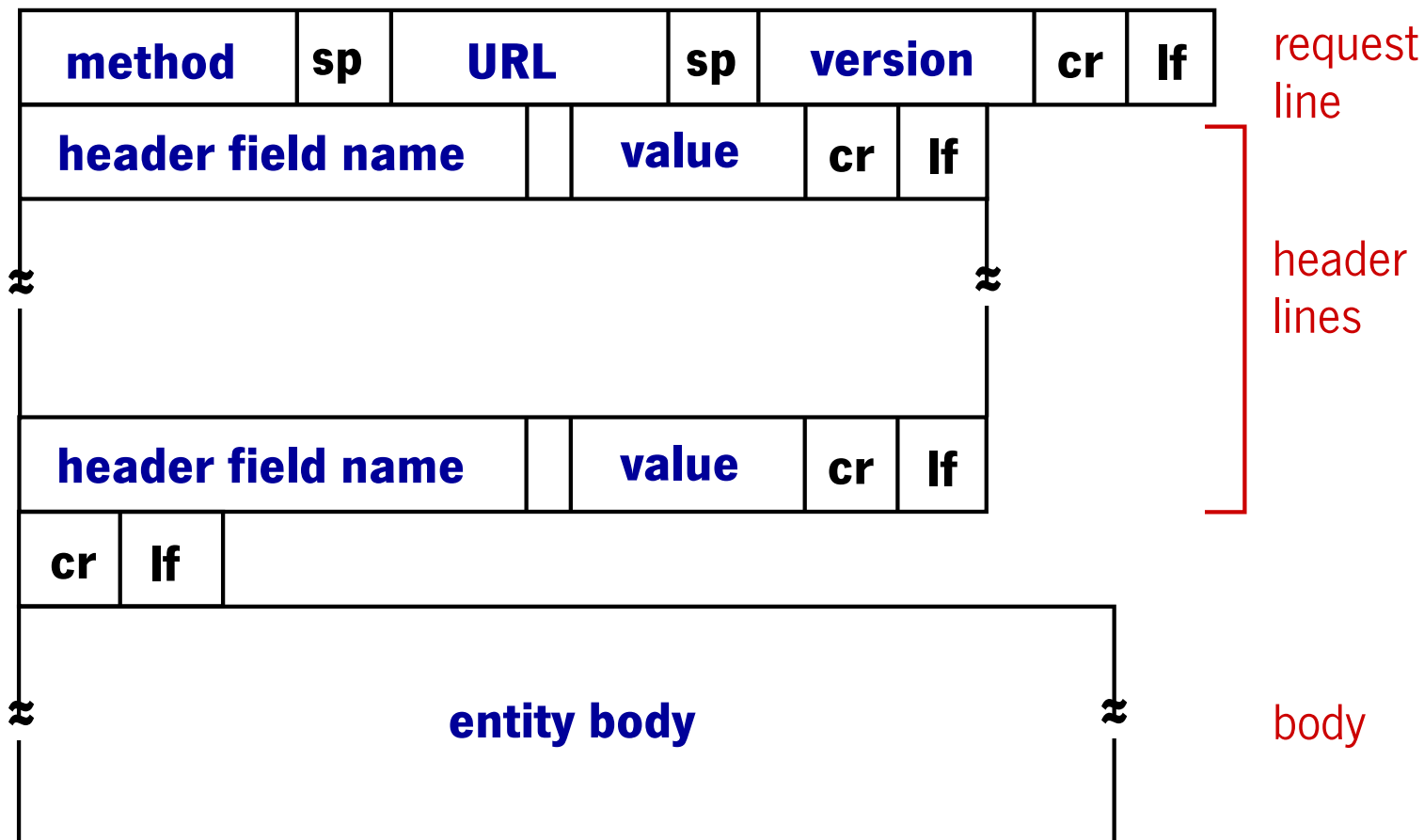


T.c.total = 468 ms (ou 488 ms contando com fecho conexão)

Aplicações de rede

Exemplo: HTTP

Formato dos PDU



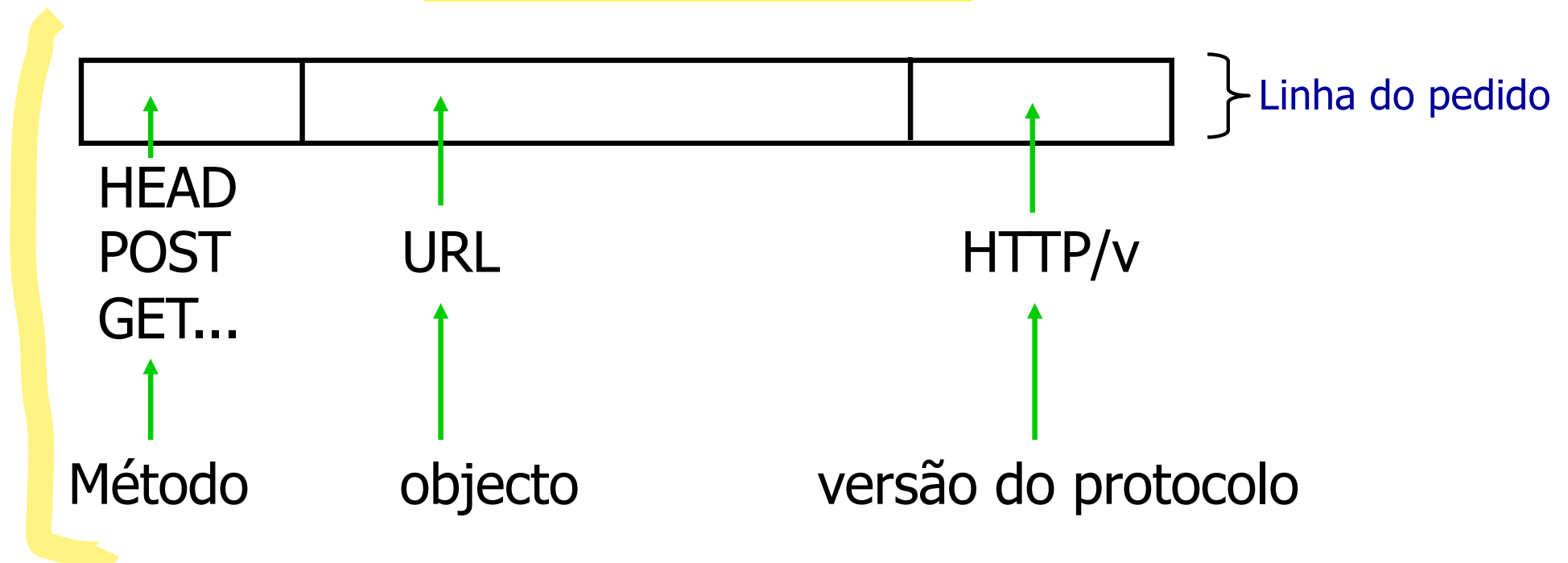


Exemplo de uma *HTTP Request Message*

GET	/directoria/pagina.html	HTTP/1.1	}	Linha do pedido
Host: www.sitio.pt				
Connection: close			}	Linhas do cabeçalho
User-Agent: Mozilla/4.0				
Accept-Language: PT				
<new line>				
Corpo da mensagem (<i>vazio no caso do GET</i>)			}	Dados da mensagem



HTTP Request Message

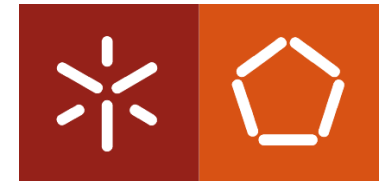


HTTP/1.0 usa conexões TCP não-persistentes:

a conexão é terminada após o envio de cada mensagem

HTTP/1.1 usa conexões TCP persistentes, por defeito

Tipos de Métodos



HTTP/1.0

- **GET**
- **POST**
- **HEAD**
 - pede ao servidor para não incluir o objeto requerido na resposta

HTTP/1.1

- **GET, POST, HEAD**
- **PUT**
 - faz o *upload* do objeto contido no corpo da mensagem na localização especificada no campo URL da mesma mensagem
- **DELETE**
 - apaga o ficheiro especificado no campo URL



Input de dados através de formulários

Método Post:

- É frequente as páginas Web incluírem um formulário para introdução de dados.
- Nesse caso pode utilizar-se o método POST em vez do método GET.
- O método POST é muito semelhante ao método GET, mas o objeto requerido depende do *input* introduzido pelo utilizador através de um formulário.
- O *Input* introduzido pelo utilizador é enviado para o servidor HTTP no corpo da *HTTP Request Message*, utilizando o método POST.

Método URL:

- Utiliza o método GET
- O Input é enviado para o servidor HTTP utilizando o campo URL da *HTTP Request Message*, com o método GET.

`www.somesite.com/animalsearch?monkeys=1&banana=4`

HOSTNAME PATH QUERY_STRING

Métodos HTTP: REST API Design



- Lista de operações sobre um **RECURSO** (ex: livros) é definida aproveitando a semântica dos métodos do protocolo HTTP:

Recurso	POST (Create)	GET (Read)	PUT (Update)	DELETE (Delete)
/livros	Cria um novo livro; Pedido: objeto “livro” no corpo do HTTP Request!	Lista todos os livros; Pedido: vazio; Resposta: listagem de livros;	Atualiza um conjunto de livros passados no corpo do pedido HTTP	Apaga todos os livros; Pedido: vazio; Resposta: sucesso ou insucesso;
/livros/01	Normalmente não é usado! Erro!	Devolve o objeto que representa o livro com id 01	Se existe livro 01 então atualiza-o; Senão dá erro!	Se existe livro 01 apaga-o;

CRUD (Create / Read / Update / Delete)

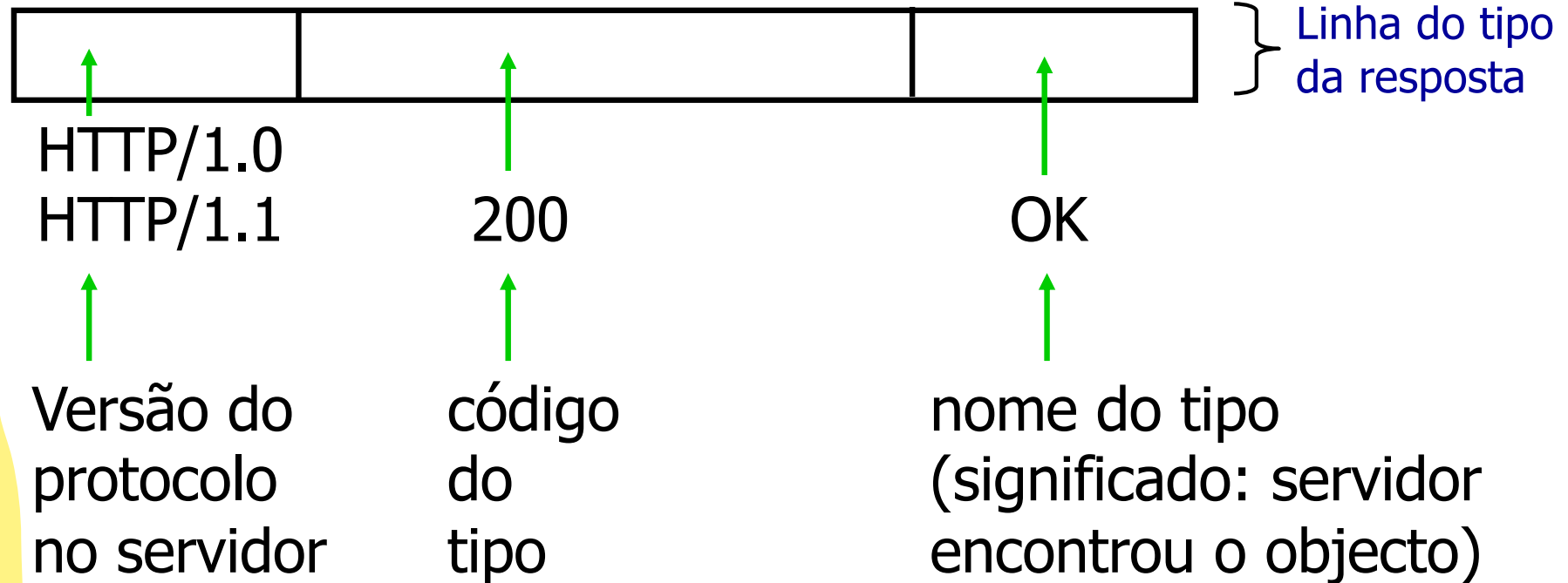


HTTP Response Message

HTTP/1.1	200	OK	} Linha do tipo da resposta
Connection: close			
Date: 07 Mai 2003 11:35:15 UTC+1			} Linhas do cabeçalho
Server: Apache/1.3.0 (Unix)			
Last-Modified: 05 Mai 2003 09:23:45 UTC+1			
Content-Length: 6825			
Content-Type: text/html			
<new line>			} Dados da mensagem
Corpo da mensagem (objecto)			



HTTP Response Message





Alguns códigos de tipo e seu significado

200 OK

301 Moved permanently, location: xyz

304 Not modified

400 Bad request (pedido não entendido)

401 Authorization required

404 Not found (objecto não encontrado)

505 HTTP version not supported

Cookies: informação de estado



A maioria dos sites Web usa cookies

Quatro componentes:

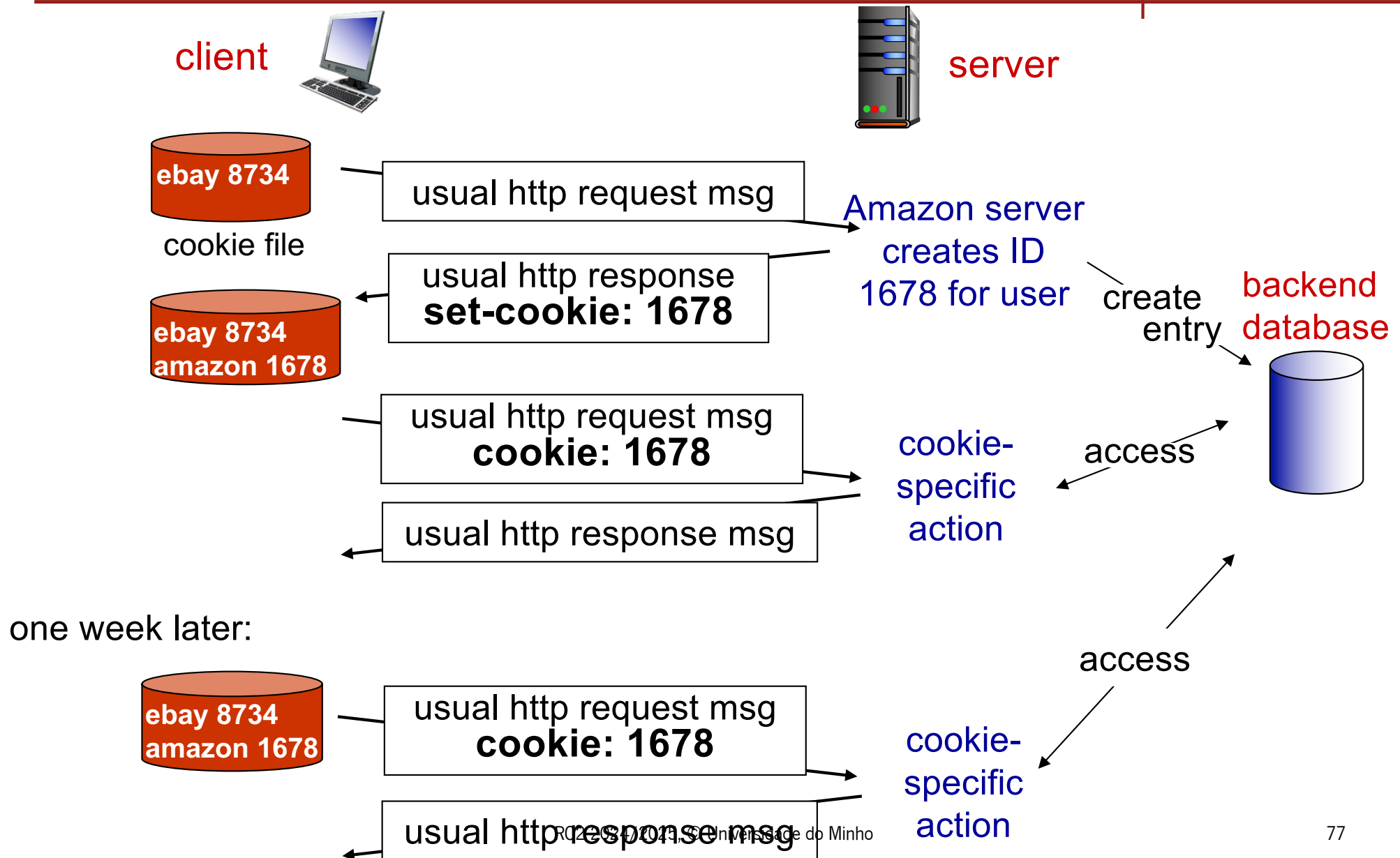
- 1) Linha com *cookie* no cabeçalho da mensagem *HTTP response*
- 2) Linha com *cookie* no cabeçalho da mensagem *HTTP request*
- 3) Ficheiro com *cookies* mantido na máquina do utilizador, gerido pelo seu browser
- 4) Uma base de dados de suporte do lado servidor Web

Exemplo:

- **Susana acede sempre à Internet a partir do seu PC**
- **visita um site de comércio electrónico pela primeira vez**
- **quando o primeiro pedido chega ao servidor Web, o servidor gera:**
 - Um Identificador (ID) único
 - Uma entrada na base de dados de suporte para esse ID



Cookies: informação de estado



Cookies: informação de estado



O que os cookies permitem:

- **autorização**
- **cabaz de compras**
- **sugestões ao utilizador**
- **informação de sessão por utilizador (ex: Web e-mail)**

efeitos colaterais

Os Cookies e a privacidade:

- **os cookies ensinam muito aos servidores a respeito dos utilizadores e seus hábitos**
- **o utilizador pode fornecer nome e e-mail ao servidor**

Como manter “estado”:

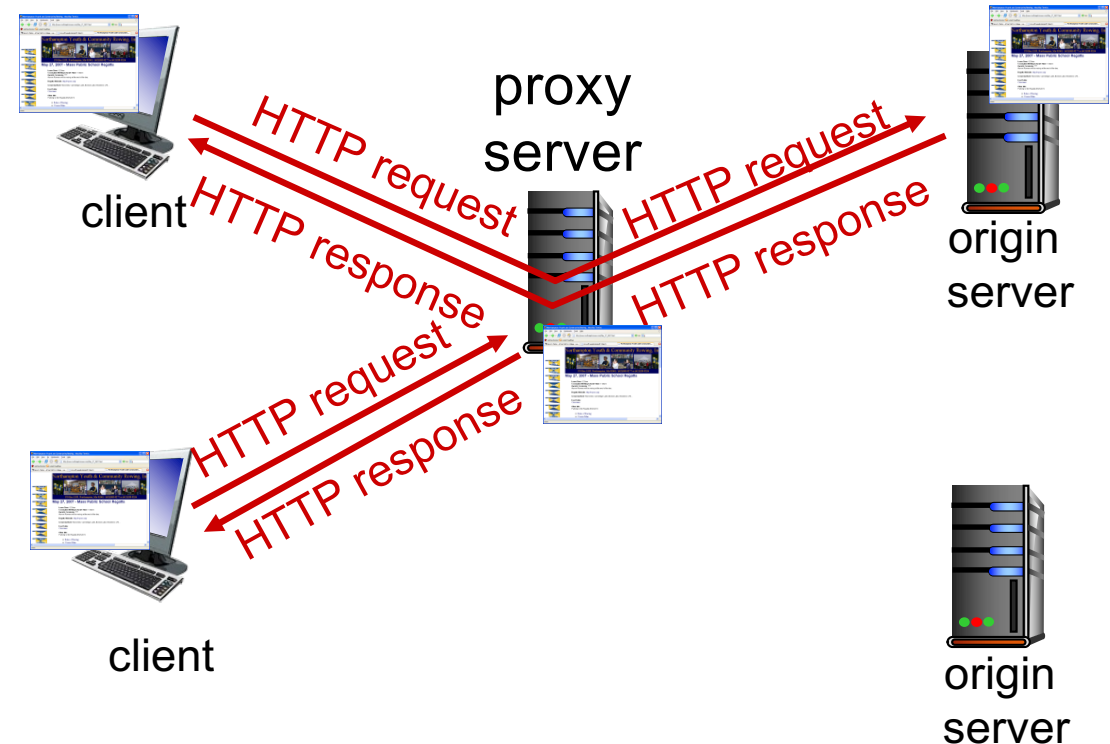
- **entidades protocolares: guardam estado por emissor/recetor entre transações distintas**
- **cookies: forma como as mensagens http transportam a informação de estado**

Web caches (servidor proxy)



Objectivo: satisfazer o pedido do cliente sem envolver o servidor HTTP alvo

- O utilizador **configura o cliente HTTP (browser)** para aceder à Web através de um servidor proxy
- O browser envia todas as **HTTP request messages** para o **servidor proxy**
 - Se o objeto requerido está na cache do proxy, o servidor proxy retorna o objeto
 - Senão o servidor proxy contacta o servidor HTTP alvo, reenvia-lhe a *HTTP request message*, aguarda a resposta que retorna ao browser



Web caching



- o **servidor proxy/cache** tem de atuar simultaneamente como cliente e como servidor
- são tipicamente instalados pelos ISP ou pelas próprias instituições (universidades, empresas, ISP residenciais, etc)

Porquê *Web caching*?

- **reduz o tempo de resposta** para os pedidos dos clientes
- **reduz o tráfego** nos links de acesso ao exterior (os mais problemáticos para a instituição).
- Internet está povoada de *caches*: permitem que fornecedores de conteúdos mais “pobres” disponibilizem efetivamente os seus conteúdos (mas isso também as redes de partilha de ficheiros P2P)



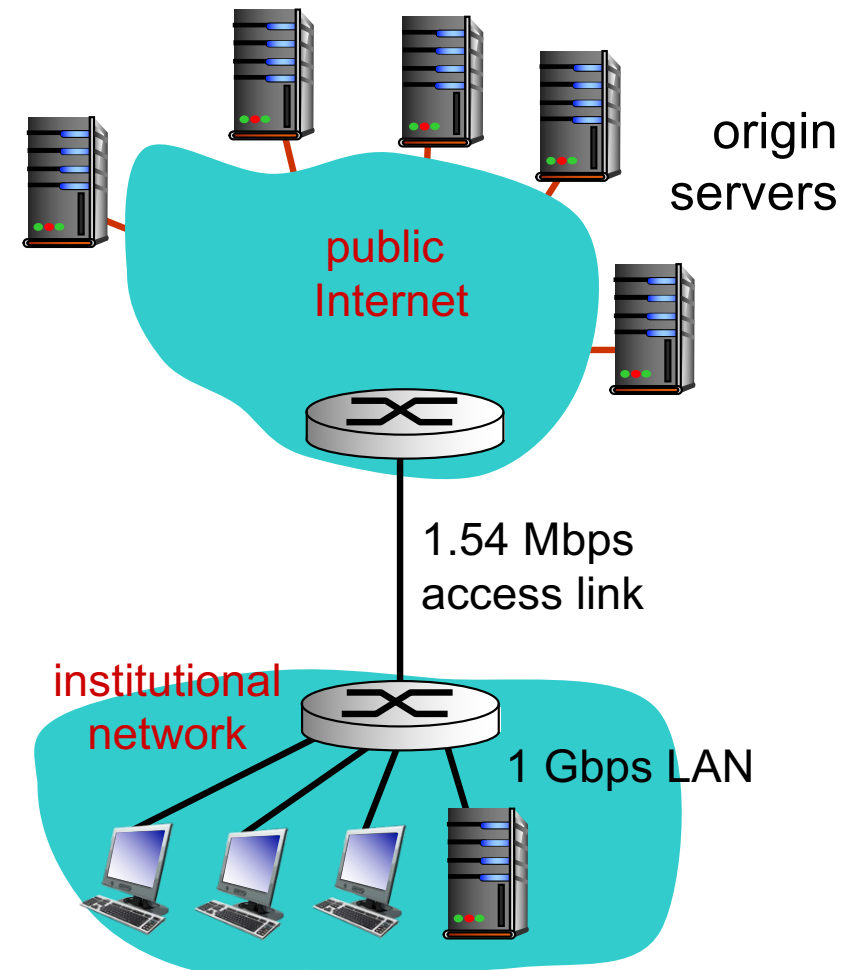
Exemplo de Caching

Pressupostos

- **Tamanho médio dos objetos = 100,000 bits**
- Taxa média de pedidos efectuados pelos *browsers* da instituição para servidores HTTP = 15/sec
- Tempo médio de atraso desde o pedido HTTP até à chegada da resposta = 2 sec

Consequências

- **Utilização da LAN = 0,15%**
(15 pedidos/sec). (100Kbits/pedido) / (1 Gbps)
- **Utilização do Link de acesso = 97%**
(15 pedidos/sec). (100Kbits/pedido) / (1.54Mbps)
- **Total delay =**
= Internet delay + access delay + LAN delay
= **2 sec + minutes + milliseconds**





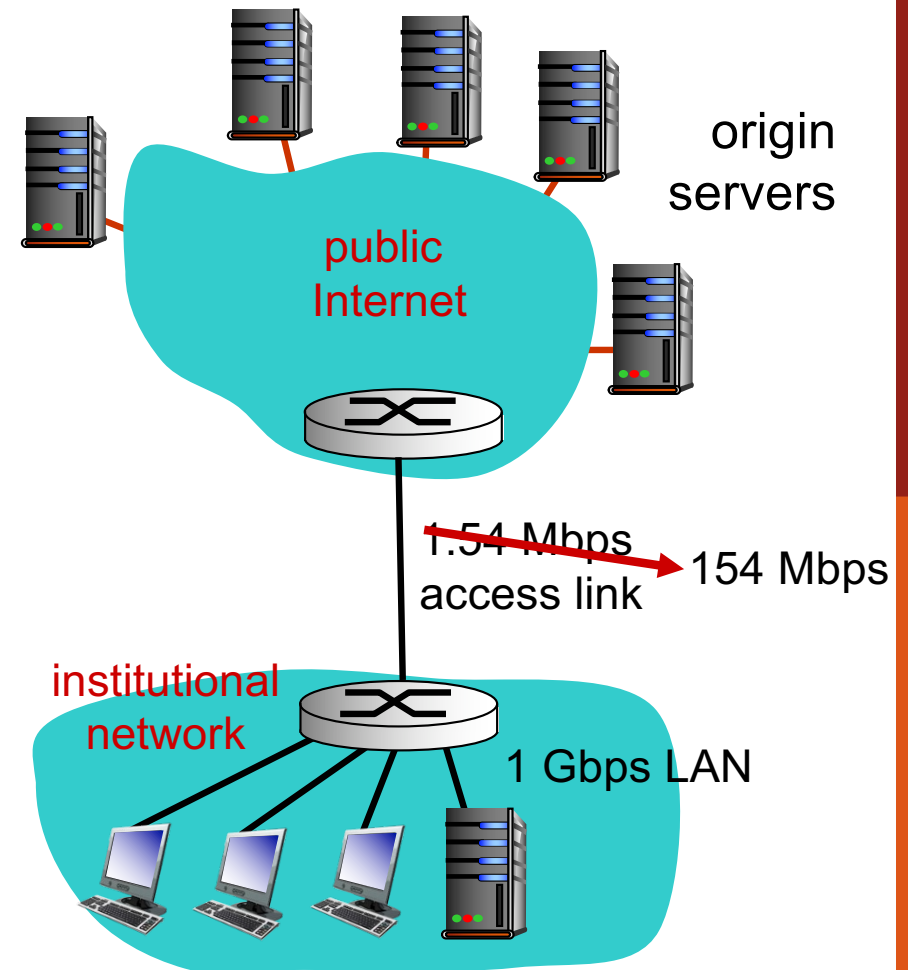
Exemplo de Caching (cont)

Solução possível

- **Aumentar a largura de banda do link de acesso para 154 Mbps**

Consequência

- Utilização da LAN = 0,15%
- Utilização do Link de Acesso = 0,97%
- Total delay = Internet delay + access delay + LAN delay
= 2 sec + msec + msec
- **É habitualmente muito dispendioso fazer o upgrade do link de acesso de uma instituição**





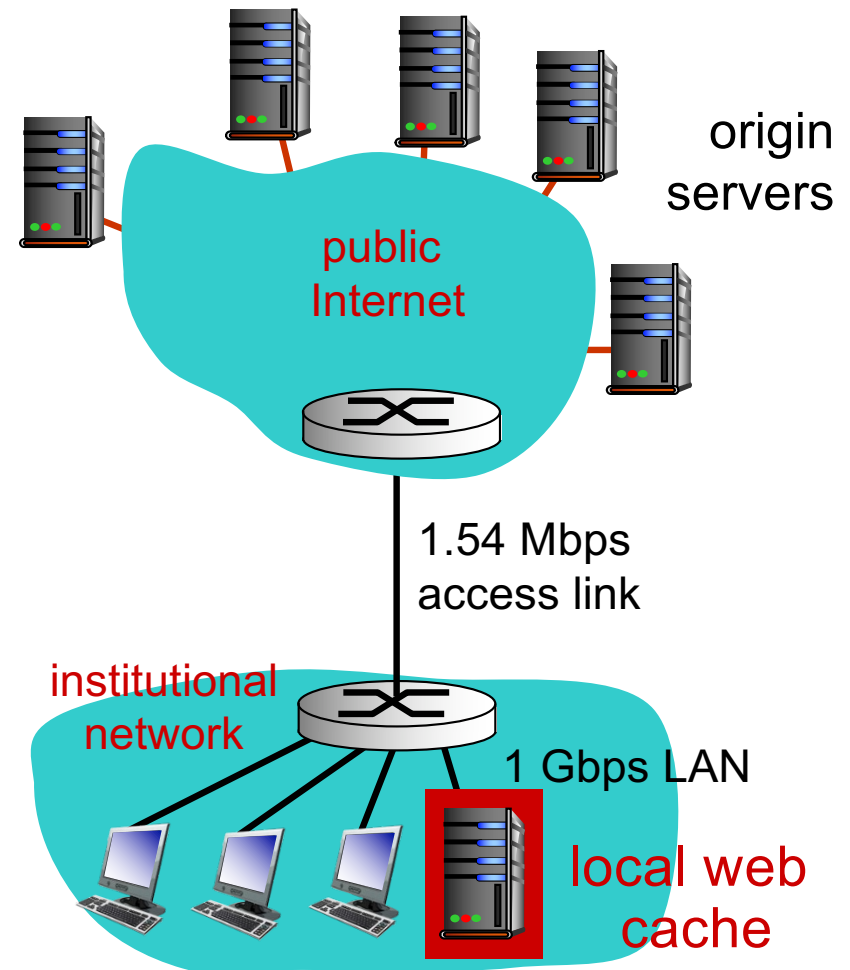
Exemplo de Caching (cont)

Solução possível: instalar o Web Proxy

- Se a taxa de acerto for de 0.4

Consequências

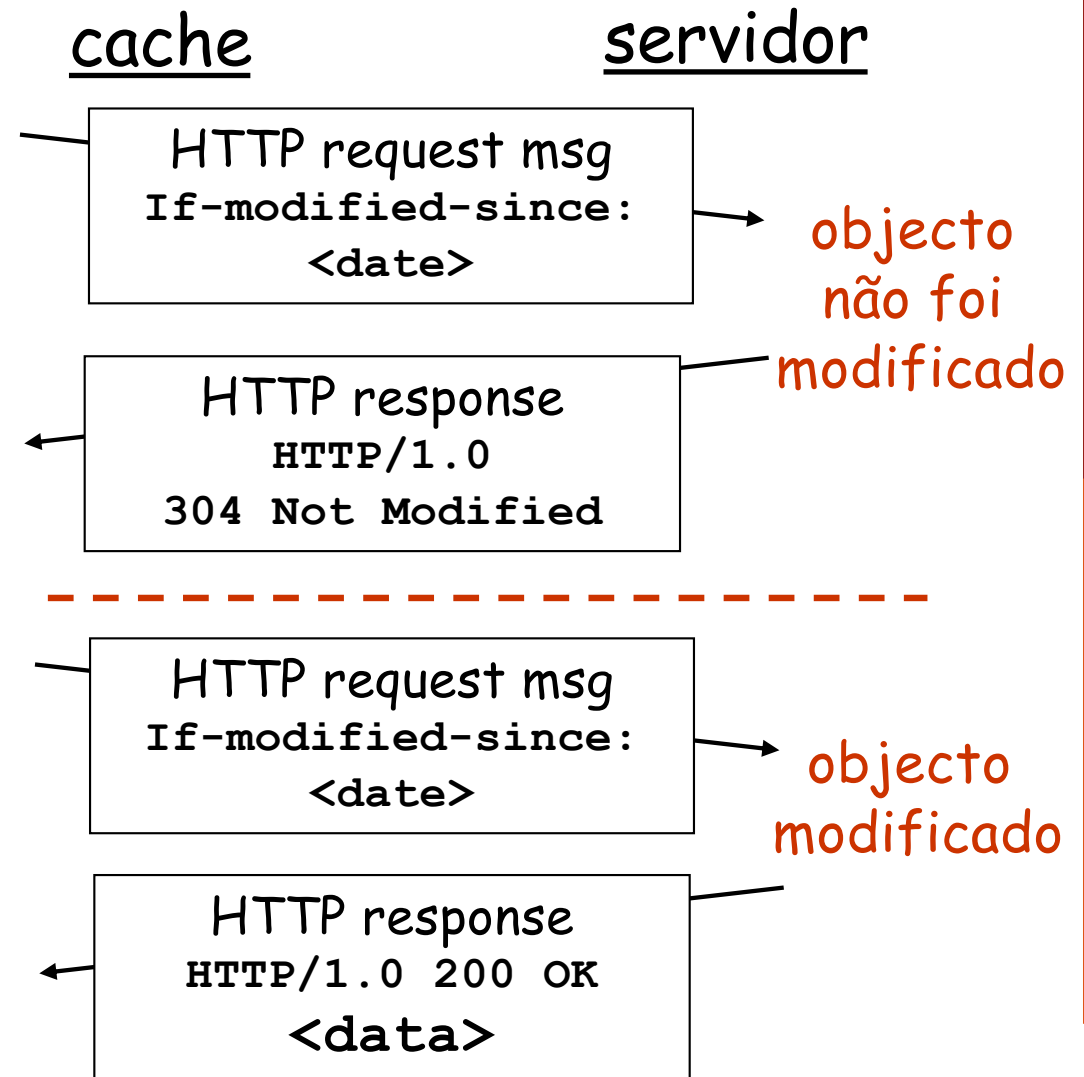
- 40% dos pedidos serão satisfeitos imediatamente
- 60% dos pedidos terão que ser redirecionados para o servidor HTTP respetivo
- A utilização do link de acesso será reduzida para 60% resultando em atrasos negligenciáveis (10 msec)
- **total avg delay =**
= Internet delay + access delay + LAN delay
= .6*(2.01) secs + .4*10msec < 1.4 secs





GET Condicional

- **Objectivo:** não enviar o objecto se a cópia mantida em cache está actualizada
- **cache:** inclui no cabeçalho do pedido HTTP, a data da cópia guardada na cache
If-modified-since:
 <date>
- **servidor:** resposta não contém nenhum objecto se a cópia mantida em cache estiver actualizada:
HTTP/1.0 304 Not Modified



Correio electrónico

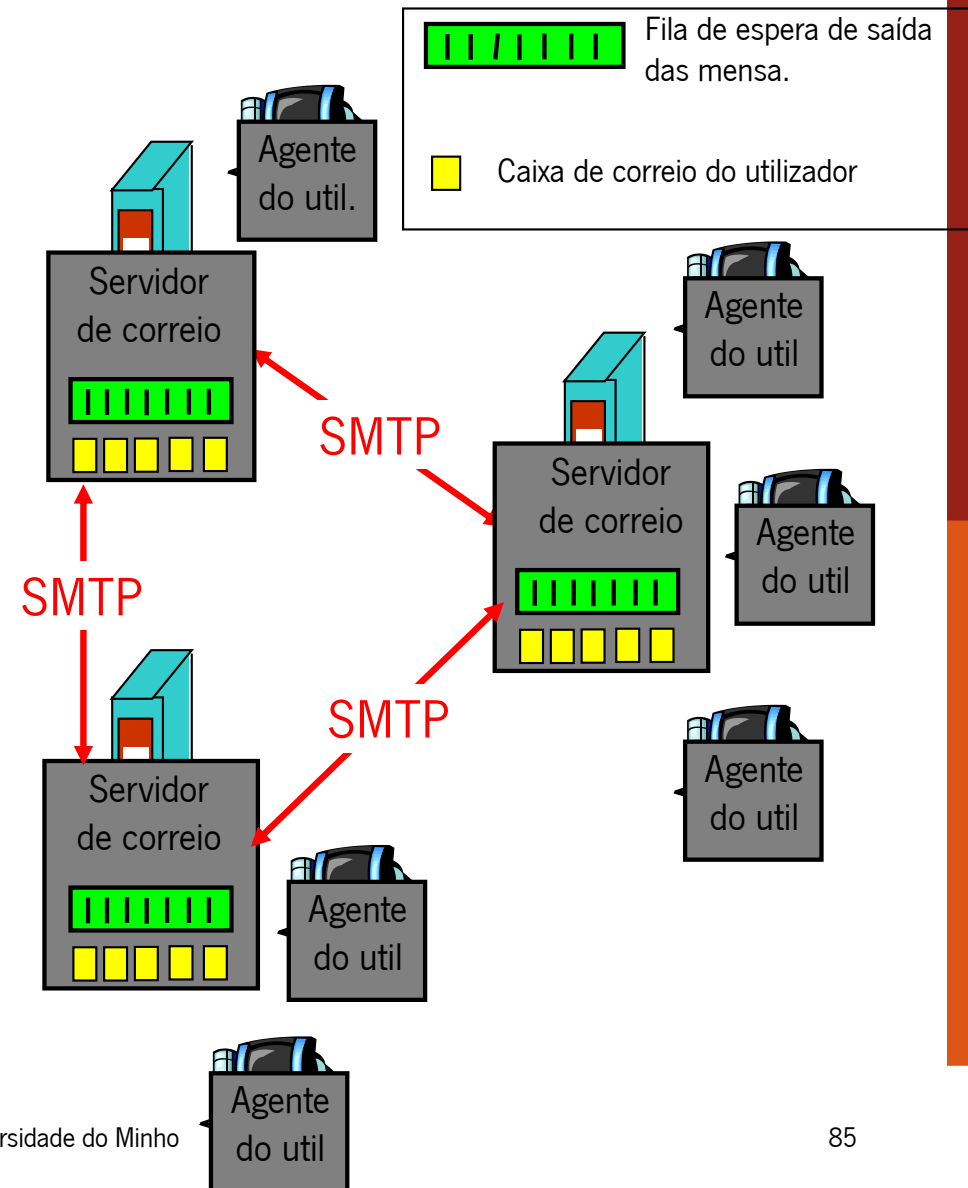


Os 3 principais componentes:

- Agentes do utilizador
- Servidores de correio
- simple mail transfer protocol: SMTP

Agente do utilizador:

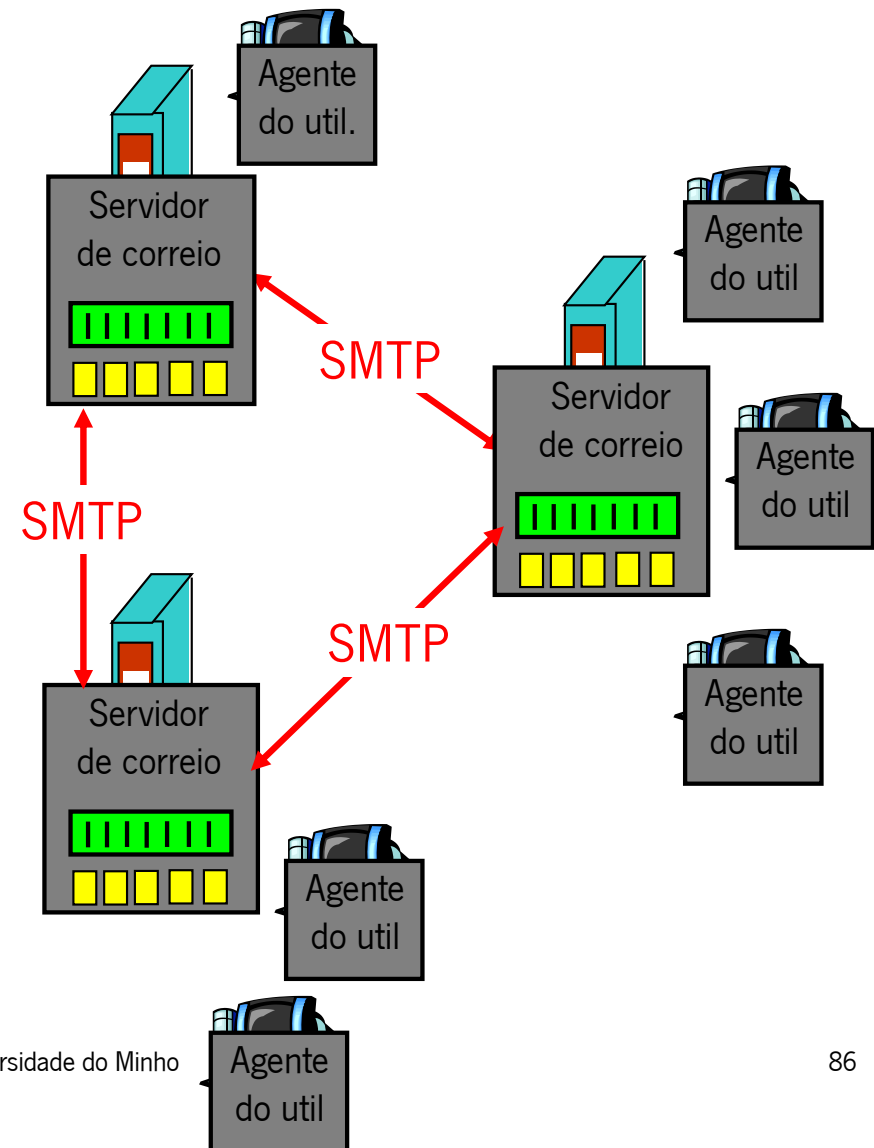
- “Faz a leitura do correio”
- composição, edição e leitura da mensagens de correio
- Ex. Eudora, Outlook, Mozilla Thunderbird
- Mostra as mensagens que estão guardadas no servidor de correio



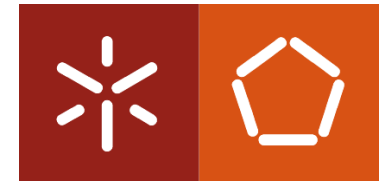
Servidor de correio electrónico



- **Caixas de correio** contêm as mensagens que se destinam aos utilizadores
- **Fila de espera das mensagens** para as mensagens que vão ser enviadas
- **Protocolo SMTP** utilizado pelo servidores de correio para trocarem mensagens de correio entre si, e para as receberem dos agentes



SMTP [RFC 2818]

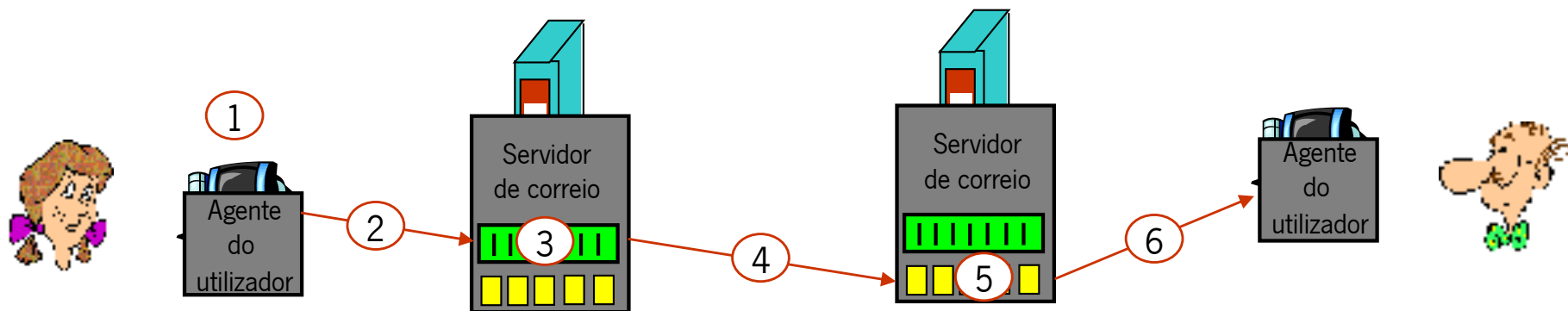


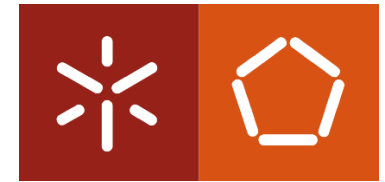
- **Usa o TCP para fazer a transmissão fiável das mensagens desde do cliente até servidor (porta 25)**
- **Transferência directa: do servidor emissor para o servidor receptor**
- **A transferência passa por 3 fases**
 - *handshaking* (cumprimento)
 - transferência das mensagens
 - término
- **Iterações com mensagens de command/response**
 - commands: texto ASCII
 - response: código e frase de estado
- **As mensagens têm de estar representadas em 7-bit ASCII**



Exemplo: mensagem da Maria para o António

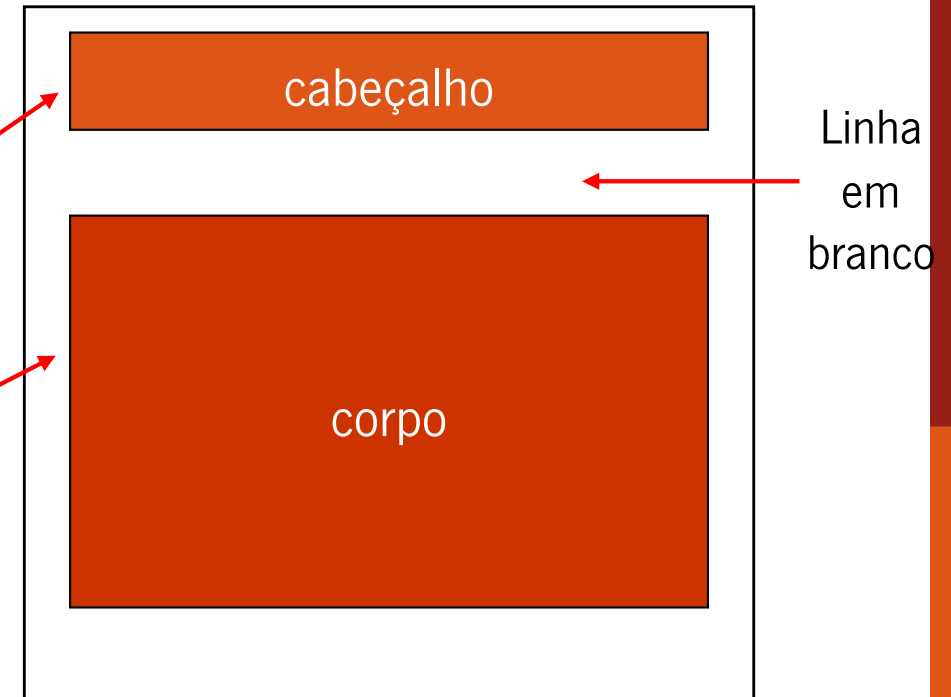
- 1) A Maria usa o agente de utilizador para escrever e enviar a mensagem para antonio@uminho.pt
- 2) O agente de utilizador da Maria envia a mensagem para o seu servidor de correio; colocando a mensagem na fila de espera
- 3) O lado de cliente do SMTP cria uma conexão TCP com o servidor de correio do António
- 4) O SMTP cliente envia a mensagem da Maria através da conexão TCP
- 5) O servidor de correio do António coloca a mensagem na caixa de correio do António
- 6) O António usa o seu agente de utilizador para ler a mensagem





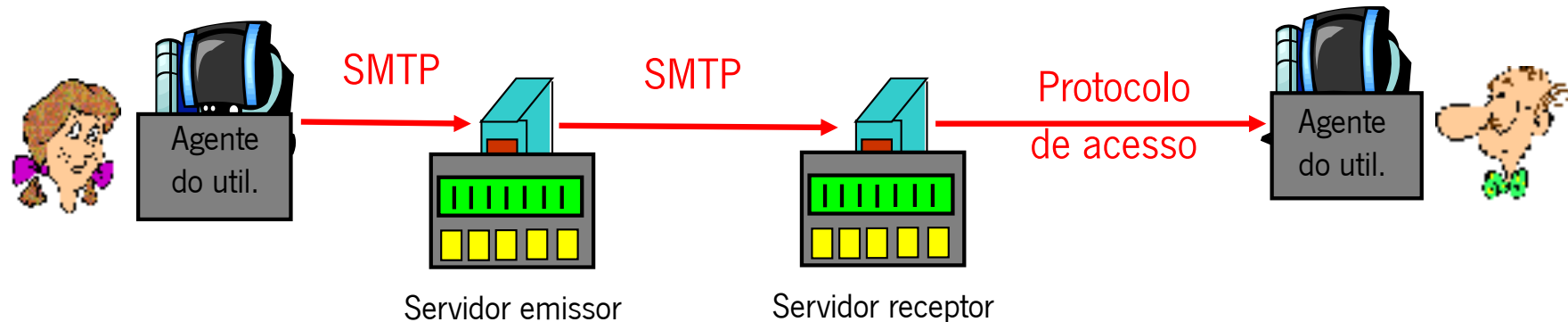
Formato das mensagens

- **SMTP: protocolo utilizado para a troca de mensagens de email**
- **RFC 822: Formato da mensagem:**
- **Linhas de cabeçalho ex:**
 - Para:
 - De:
 - Assunto:
- **Corpo**
 - O texto da mensagem”, em caracteres ASCII





Protocolos de acesso ao correio electrónico



- **SMTP: Entrega/armazenamento no servidor receptor**
- **Protocolo de acesso ao correio: entrega das mensagens do servidor receptor ao agente do utilizador**
 - POP: Post Office Protocol [RFC 1939]
 - Autorização (agente $\leftrightarrow$ servidor) e download
 - IMAP: Internet Mail Access Protocol [RFC 1730]
 - mais funcionalidades (mais complexo)
 - manipulação das mensagens armazenadas no servidor
 - HTTP: gmail, Hotmail, Yahoo! Sapo, etc.

Protocolo POP3



Fase autorização

- **Comandos do cliente:**
 - Utilizador: nome
 - Palavra-passe: password
- **Respostas do servidor**
 - +OK
 - -ERR

```
S: +OK POP3 server ready
C: user António
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

Fase de transferência, cliente:

- **list: lista o número de mensagens**
- **retr: entrega a mensagem**
- **dele: elimina**
- **Quit: sair**

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```



DNS: *Domain Name System*

Pessoas: usam muitos identificadores:

- BI, NIF, passaporte

Internet: sistemas terminais e routers usam:

- Endereços IP (32 bit) – são usados para endereçar datagramas
- “nomes”, ex, www.uminho.pt - são usados pelos humanos

Q: Como se mapeiam os endereços IP nos nomes?

Domain Name System:

- Base de dados distribuída implementada numa hierarquia de muitos servidores de nomes



Serviços DNS:

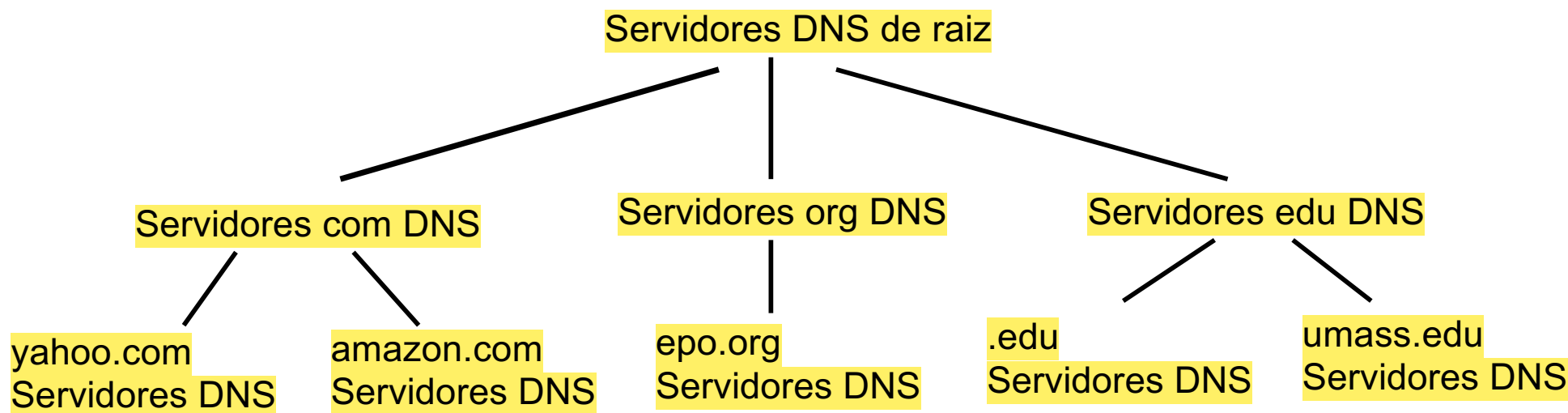
- **hostname:**
 - para representar o endereço IP
- **host aliasing:**
 - canónico, pseudónimos
- **mail server aliasing**
- **distribuição da carga:**
 - servidores web replicado: conjunto de endereços IP para um único nome canónico

Porque não se utiliza um DNS centralizado?

- **um único ponto de falha**
- **volume de tráfego**
- **distância da base de dados centralizada**
- **manutenção**

Portanto não escala!

DNS: base de dados hierárquica e distribuída



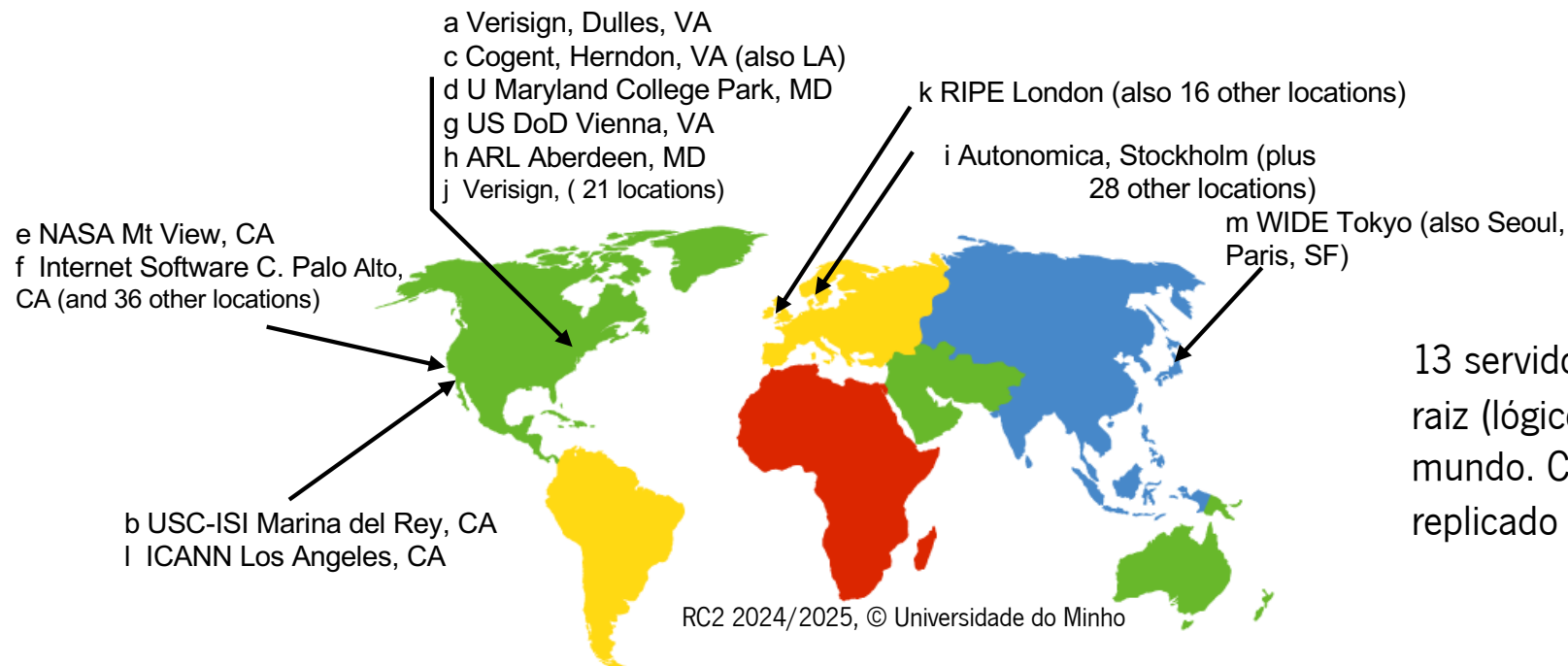
O Cliente pretende o IP para `www.amazon.com`; 1ª aproximação:

- **O cliente pede ao servidor de raiz para encontrar o servidor DNS `.com`**
- **O cliente pede ao servidor DNS `.com` para obter o servidor DNS `amazon.com`**
- **O cliente pede ao servidor DNS `amazon.com` para obter o endereço IP do nome `www.amazon.com`**

DNS: servidores de raiz



- **São contactados pelos servidores locais quando estes não conseguem resolver os nomes**
- **servidor de nomes raiz:**
 - Contactam os servidores com autoridade para obter o mapeamento do nome em endereço, que retornam para o servidor local



13 servidores de nomes raiz (lógicos), em todo mundo. Cada um deles replicado muitas vezes!

DNS: TLD e servidores autorizados



- **Top-level domain (TLD) servers:**

- são responsáveis por com, org, net, edu, etc, e pelos domínios dos países pt, uk, fr, ca, jp, ...

- **Servidores de DNS com autoridade:**

- servidores DNS de organizações que fornecem o mapeamento do hostname para IP para os servidores da organização (ex. Web, mail).
- podem ser mantidos pela organização ou pelo ISP desta.

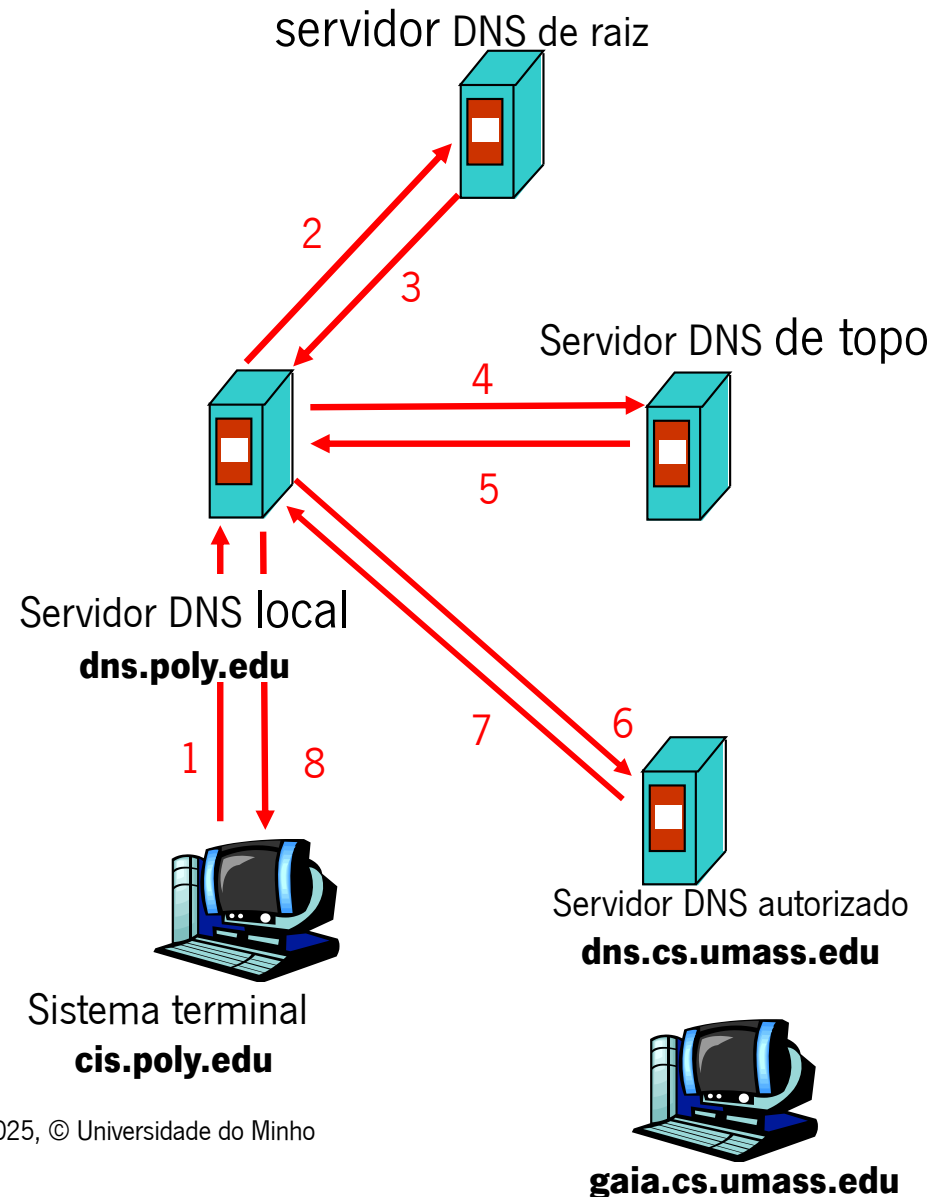
- **Servidores de DNS Locais:**

- Não pertencem à hierarquia mas atuam como “proxies”
- Cada ISP tem o seu. Quando um sistema terminal faz uma query esta é encaminhada para o servidor local que a encaminha para a hierarquia do DNS.

DNS: resolução de nomes



- **O sistema terminal com o nome cis.poly.edu pretende o endereço IP do nome gaia.cs.umass.edu**
- **Pedido iterativo:**
 - o servidor contactado retorna o nome do servidor que deverá ser contactado: “Eu não sei este nome, mas pergunta a este servidor”

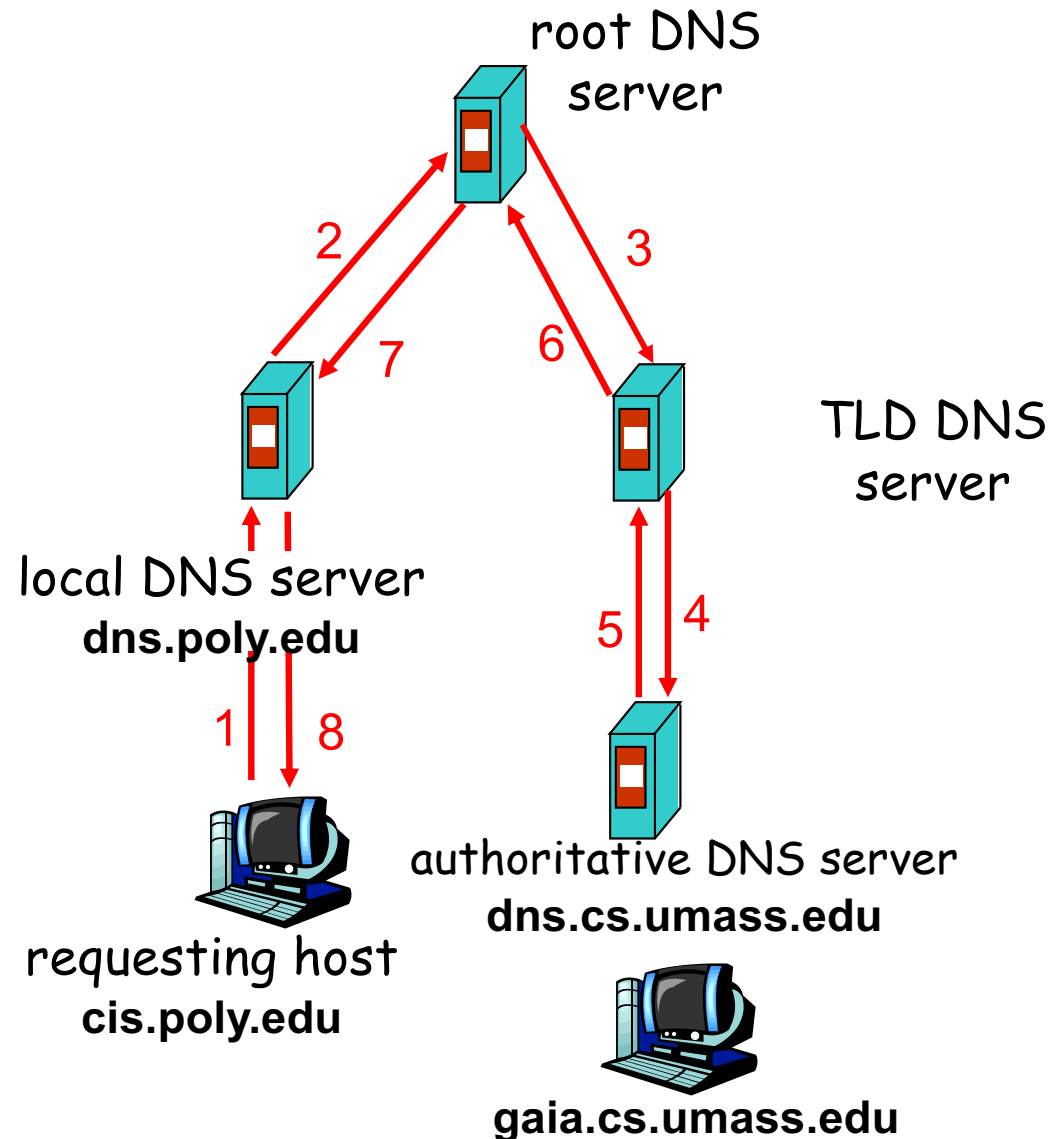


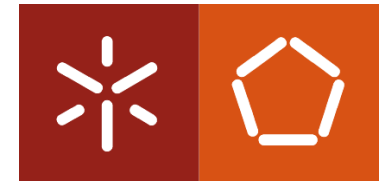
DNS: resolução de nomes



- **Pedido recursivo**

- o servidor contactado contacta com outros servidores para retornar a resposta pretendida

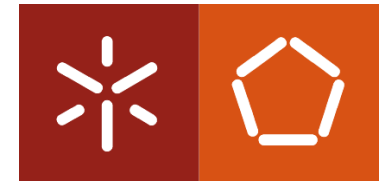




DNS: *cache* e actualização dos registos

- **Depois de aprender o mapeamento, o servidor de nomes guarda em *cache* o mapeamento**
 - os registos em *cache* expiram passado algum tempo e são eliminados
 - os servidores TLD normalmente estão na *cache* dos servidores de nomes locais
 - evitando desta forma os pedidos aos servidores raiz

DNS: registos



DNS: Base de dados distribuída que armazena registos de recursos - resource records (RR)

Formato RR: (nome, valor, tipo, ttl)

☐ Tipo=A

- ❖ **Nome** é o nome do sistema terminal
- ❖ **Valor** é o endereço IP

☐ Tipo=NS

- ❖ **Nome** é o domínio (ex. uminho.pt)
- ❖ **Valor** é o nome do sistema terminal que está autorizado a ser o servidor de nomes deste domínio

☐ Tipo=CNAME

- ❖ **Nome** é o pseudónimo.
- ❖ **Valor** é o nome canónico
Por ex: www.ibm.com é realmente servereast.backup2.ibm.com

☐ Tipo=MX

- ❖ **valor** é o nome do servidor de correio associado com o **Nome**